



PROGETTO DI NATURAL LANGUAGE PROCESSING

Fine-Tuning e valutazione di Large Language Model per
la generazione del dialetto napoletano: Traduzione,
Completamento di Dialogo e Generazione libera a
partire dal linguaggio parlato spontaneo

Professori

Prof.ssa Loredana Caruccio

Prof.ssa Grazia Margarella

Studenti

Chiara Puglia, 0522501984

Luca Giuliano, 0522501931

Università degli Studi di Salerno
a.a 2025/2026

Indice

1	Stato dell'Arte	3
1.1	Introduzione	4
1.2	Dataset	4
1.3	Golden Standard	6
1.4	Dataset Finale	8
2	Fine-Tuning attraverso l'utilizzo del modello Minerva	8
2.1	Definizione del task relativo alla Traduzione	9
2.2	Continued Pre-Training sul Napoletano	10
2.3	Iniezione Lessicale	11
2.4	Fine-Tuning per Traduzione	12
2.5	Analisi dei risultati	13
2.6	Fine-Tuning per Completamento di dialogo	15
2.7	Analisi dei Risultati	17
2.8	Fine-Tuning per Generazione Libera	20
2.8.1	Analisi dei Risultati	21
3	Fine-Tuning attraverso l'utilizzo del modello LLama	22
3.1	Analisi dei Risultati sulla Traduzione	22
3.2	Analisi dei Risultati sul Completamento di dialogo	26
3.3	Analisi dei Risultati sulla Generazione Libera	31
4	Fine-Tuning attraverso l'utilizzo del modello Gemma	32
4.1	Analisi dei Risultati sulla Traduzione	32
4.2	Analisi dei Risultati sul Completamento di dialogo	37
4.3	Analisi dei Risultati sulla Generazione Libera	41
5	Valutazione umana su scala Likert	41
5.1	Scelta dei sistemi da valutare	42
5.2	Protocollo di annotazione	42
5.3	Analisi dei risultati	44
6	Conclusioni	50
7	Sviluppi Futuri	51

1 Stato dell'Arte

Il progetto nasce con l'obiettivo di addestrare diversi modelli Large Language Model il cui scopo è quello di comprendere e produrre il dialetto napoletano, non come traduzione meccanica dall'italiano ma come lingua viva capace di adattarsi al contesto della conversazione: il sistema dovrà infatti essere in grado di tradurre correttamente enunciati dall'italiano al napoletano, di proseguire in modo coerente una conversazione già avviata in dialetto e, soprattutto, di generare liberamente risposte in napoletano che rispettino il registro, il lessico e il tono della conversazione in corso, così come farebbe un parlante nativo che alterna spontaneamente italiano e dialetto nel parlato quotidiano. Poiché il napoletano, come la maggior parte dei dialetti italiani, non dispone di risorse testuali ampie e standardizzate su cui addestrare un modello linguistico, si è scelto di partire da conversazioni reali, spontanee e contestualizzate, raccolte sul campo in Campania, in modo da catturare un uso autentico del dialetto invece di un napoletano decontestualizzato.

Il lavoro svolto finora consiste nell'individuazione e nell'analisi approfondita della risorsa linguistica su cui basare questo addestramento: il modulo KIPasti [1] del corpus KIParla, realizzato dall'Università di Bologna nell'ambito del progetto DiverSIta — Diversity in Spoken Italian, finanziato dal MUR (bando PRIN 2022 PNRR). Il modulo raccoglie oltre 40 ore di parlato spontaneo, registrato tra il 2020 e il 2024 in tredici regioni italiane durante conversazioni conviviali prevalentemente in ambito familiare, per un totale di 63 conversazioni e 145 parlanti; l'italiano è la lingua predominante in tutte le interazioni, ma nella maggioranza di esse (78%) sono presenti anche passaggi in dialetto. Ogni conversazione è disponibile in quattro formati paralleli — file ELAN allineati temporalmente, trascrizione lineare in stile Jefferson (che annota anche intonazione, pause, sovrapposizioni, volume e velocità dell'eloquio), trascrizione lineare puramente ortografica e dati verticalizzati a livello di singolo token — ed è corredata da metadati dettagliati su parlanti (età, genere, regione di nascita, occupazione, titolo di studio, prima lingua) e conversazioni (durata, numero di partecipanti, lingue parlate, luogo e anno di raccolta). Particolarmente utile ai fini del progetto è la colonna *variation* del formato verticalizzato, che segnala automaticamente le unità di trascrizione in cui è presente commutazione di codice con il dialetto, rendendo possibile isolare in modo mirato i passaggi in napoletano.

All'interno del corpus è stato individuato il sottoinsieme di conversazioni più rilevante per l'apprendimento del napoletano: cinque conversazioni raccolte a Caserta nel 2020 (codici KPS021–KPS025), per un totale di circa 3 ore e 37 minuti di parlato e 9 persone coinvolte, tutte contenenti passaggi in dialetto. Due dei parlanti coinvolti, entrambi anziani, hanno il dialetto come prima lingua, rendendo una di queste conversazioni una fonte particolarmente ricca di napoletano autentico e non mediato dall'italiano. Trattandosi di materiale già trascritto, anonimizzato e liberamente distribuito con licenza Creative Commons (CC BY-NC-SA 4.0), questo sottoinsieme rappresenta una base testuale immediatamente utilizzabile, solida sul piano metodologico e coerente con l'obiettivo di far apprendere al modello un dialetto vivo, situato e contestualizzato.

A partire da questa base, il progetto si propone ora di estrarre ed elaborare in modo mirato i

passaggi in napoletano presenti nelle conversazioni campane, sfruttando anche l'allineamento tra trascrizione ortografica e forma dialettale per costruire materiale di addestramento utile sia alla traduzione italiano-napoletano sia alla generazione libera di risposte in dialetto, e integrare questo materiale nella pipeline affinché quest'ultimo sia in grado non solo di tradurre, ma anche di continuare naturalmente una conversazione già avviata in napoletano e di generare autonomamente risposte dialettali coerenti con il contesto, il tono e il registro dello scambio in corso, riproducendo così la naturalezza e la variabilità con cui il dialetto viene realmente parlato in Campania.

1.1 Introduzione

A partire dalla risorsa linguistica descritta nello stato dell'arte, il presente lavoro introduce la simulazione di un sistema conversazionale in dialetto napoletano capace di tradurre correttamente enunciati dall'italiano al napoletano, di proseguire in modo coerente una conversazione già avviata in dialetto e di generare liberamente risposte che rispettino il registro, il lessico e il tono del contesto conversazionale, così come farebbe un parlante nativo.

Per raggiungere questo obiettivo, il seguente progetto prevede lo svolgimento dei seguenti punti: Innanzitutto verrà effettuata l'analisi esplorativa del dataset Kipasti di partenza, andando a selezionare 2 conversazioni totalmente in italiano delle 65 conversazioni totali ottenendo in questo modo un dataset di 2589 righe. Nello step successivo, utilizzeremo un LLM che effettuerà la traduzione in napoletano delle conversazioni in italiano. I risultati ottenuti dall'LLM saranno successivamente confrontati con un golden-standard che prevede la traduzione manuale delle conversazioni in Napoletano attraverso l'utilizzo di una piattaforma [2] e di metriche di contesto, quali **Bert-Score**, **Bleu**, **Rouge** e **chrF medio**.

Una volta effettuata la traduzione manuale ed effettuato il confronto con la traduzione effettuata dall'LLM, le frasi che non sono state tradotte correttamente dall'LLM in Napoletano verranno sostituite con quelle tradotte manualmente in modo da evitare incongruenze. Lo step successivo riguarda la costruzione di un prompt che verrà utilizzato per l'addestramento che prevede lo svolgimento dei seguenti task: Traduzione Italiano-Dialetto, completamento di dialogo e generazione libera ognuno dei quali verrà trattato singolarmente e per ognuno di essi, verranno calcolate metriche come Accuracy, Precision, Recall, F1-Score. Successivamente verrà calcolata la scala Likert che effettuerà la valutazione sulla naturalezza dialettale, accuratezza grammaticale e coerenza lessicale. Infine verranno fornite le conclusioni che riassumeranno gli obiettivi che sono stati raggiunti ai quali seguiranno eventuali sviluppi futuri.

1.2 Dataset

Il dataset che è stato preso in considerazione per questo lavoro è il dataset KIPasti, il quale è composto da 63 conversazioni. Di queste, 13 sono completamente in italiano, mentre le restanti 50 contengono anche tratti di dialetto in code-mixing con l'italiano. Le 13 conversazioni solo in italiano sono: KPC003, KPC005, KPC008 (Toscana/Marche,

centro), KPN001, KPN003, KPN004, KPN007, KPN009, KPN026, KPN027, KPN029 (Emilia-Romagna, nord), KPS005 (Basilicata, sud) e KPS007 (Abruzzo, sud).

Il corpus nel suo complesso raccoglie oltre 40 ore di parlato spontaneo, registrato tra il 2020 e il 2024 in tredici regioni italiane durante conversazioni conviviali (perlopiù in ambito familiare), per un totale di 145 parlanti coinvolti. Ogni conversazione presenta le seguenti caratteristiche:

- **code:** codice identificativo univoco della conversazione (es. KPS021).
- **type:** tipo di interazione; in questo modulo vale sempre free-conversation, cioè conversazione libera non guidata.
- **duration:** durata della conversazione, nel formato hh:mm:ss.
- **participants-number:** numero di parlanti coinvolti in quella conversazione.
- **participants:** elenco dei codici dei parlanti che vi hanno preso parte (collegabili alla tabella participants.tsv).
- **year:** anno in cui la conversazione è stata registrata (tra il 2020 e il 2024).
- **collection-point:** codice di due lettere del punto di raccolta (es. CE per Caserta).
- **collection-region:** regione italiana in cui si trova il punto di raccolta (ad esempio, Campania).
- **macro-region:** macro-area geografica di appartenenza: nord, centro o sud.
- **languages:** lingue effettivamente parlate nella conversazione: italian (solo italiano) oppure italian;dialect (italiano con passaggi in dialetto).
- **participants-relationship:** tipo di relazione tra i parlanti; in questo modulo è sempre symmetric, cioè un rapporto simmetrico tra pari (tipicamente familiari/conoscenti, non un'intervista gerarchica).
- **moderator:** indica se era presente un moderatore che guidava la conversazione (yes/no); nel modulo KIPasti è sempre no dato che si tratta di conversazioni spontanee.
- **topic:** argomento della conversazione; per questo modulo è lasciato libero (free), dato che si tratta di chiacchiere spontanee durante il pasto e non di conversazioni a tema.

A causa della limitata potenza di calcolo a disposizione, per evitare eventuali problematiche negli step successivi, è stata effettuata una riduzione del dataset in cui sono state riportate interamente 2 conversazioni, rispettivamente KPN001 e KPN003 interamente in italiano per un totale di 2589 righe. Ognuna di queste conversazioni presentano le seguenti caratteristiche:

- **id:** codice identificativo della conversazione, coerente con quello utilizzato nel corpus KIPasti (es. KPN001).

- **regione:** regione italiana di raccolta della conversazione (in questo caso specifico emilia-romagna).
- **macro_regione:** macro-area geografica di appartenenza (in questo caso sempre nord).
- **languages:** lingua/e parlate nella conversazione originale; in questo file vale sempre italian, coerentemente con la selezione delle conversazioni monolingue.
- **n_turni_italiani:** numero totale di turni in italiano presenti in quella conversazione (corrisponde esattamente alla lunghezza della lista turni).
- **turni:** lista dei singoli turni di parola che compongono la conversazione, ciascuno rappresentato come oggetto a sé.

Ogni turno a sua volta è costituito da:

- **turn_index:** numero progressivo del turno all'interno di questa lista (parte da 1).
- **tu_id:** identificativo originale della transcription unit nel corpus KIPasti (permette di ricollegare il turno alla fonte originale).
- **speaker:** codice del parlante che ha pronunciato il turno (oppure ??? quando il parlante non è identificabile, come da convenzione del corpus).
- **italiano:** testo del turno nella forma originale in italiano.

In questa fase, è stata aggiunta un'ulteriore colonna all'interno del dataset, ovvero la colonna «napoletano», che contiene la traduzione della frase presente nella colonna «italiano». La traduzione è stata effettuata da gemma4:cloud [3], un LLM hostato da OLLAMA e richiamato tramite una chiave privata.

1.3 Golden Standard

In seguito a quanto descritto nel paragrafo precedente, è stato realizzato un golden standard per verificare la correttezza del testo generato dal modello; a tal proposito, è stato utilizzato il traduttore online italiano-napoletano [2]. Le frasi in napoletano generate dal modello sono state confrontate con quelle del golden standard mediante BERTScore, BLEU, ROUGE e chrF. Per ogni turno, il sistema combina le quattro metriche in un punteggio di similarità complessivo, attribuendo un peso maggiore a chrF e BERTScore, che rilevano meglio le somiglianze a livello di caratteri e di significato, e un peso minore a BLEU e ROUGE, più rigide. Se la somiglianza scende sotto l'85%, il turno viene contrassegnato come «da sostituire» con la versione del golden standard; in caso contrario, viene mantenuta la traduzione in napoletano prodotta dal modello.

In seguito è stato riportato un'estratto di quelli che sono stati i risultati ottenuti:

```
=====
COERENZA TRADUZIONI NAPOLETANE --- gemma4:cloud vs golden validato a mano
=====
Ipotesi (candidato) : results/napoletano.json
Riferimento (gold)  : golden_translate\golden_validator.json
```

```

Turni confrontati      : 2568
BERTScore              : bert-base-multilingual-cased
chrF                   : word_order=0
Pesi similarita'       : chrF 40%, bertscore 30%, bleu 15%, rouge 15%
Soglia booleano        : sostituire = similarita' < 85.0%

```

MEDIE SULL'INTERO INSIEME

```

BERTScore F1 medio : 0.8605
BLEU medio (frase) : 38.57
ROUGE-L F1 medio   : 0.6455
chrF medio (frase) : 62.20
BLEU di corpus     : 31.35
chrF di corpus     : 61.66
Similarita' media  : 66.16 \%
Da sostituire      : 2053/2568 (79.9%) --- identici (100%): 436

```

DETTAGLIO PER TURNO

[KPN001 | turno 1 | tu_id 0 | speaker PKP135]

```

Italiano              : emh
Frase dialetto gemma4:cloud : ehm
Frase dialetto golden  : emh
Bert-Score (F1)       : 0.8749
BLEU                  : 0.00
Rouge (ROUGE-L F1)    : 0.0000
ChrF                  : 33.33
Grado di similarita'   : 39.58 \%
Sostituire con golden  : True

```

[KPN001 | turno 2 | tu_id 1 | speaker PKP007]

```

Italiano              : tu non hai mai fatto queste foto quando sei andata in barca
Frase dialetto gemma4:cloud : tu nun ha' maje fatto sti fote quanno sia' ghiuta 'n varca
Frase dialetto golden  : tu nun l'he maje fatte sti ffoto quanno si' ghiuta 'ncopp'a varca
Bert-Score (F1)       : 0.8239
BLEU                  : 9.26
Rouge (ROUGE-L F1)    : 0.5833
ChrF                  : 51.13
Grado di similarita'   : 55.31 \%
Sostituire con golden  : True

```

[KPN001 | turno 3 | tu_id 2 | speaker PKP135]

```

Italiano              : no direi di no
Frase dialetto gemma4:cloud : no, nun credo proprio
Frase dialetto golden  : no, dicesse 'e no
Bert-Score (F1)       : 0.7241
BLEU                  : 21.36
Rouge (ROUGE-L F1)    : 0.2500
ChrF                  : 13.85
Grado di similarita'   : 34.22 \%
Sostituire con golden  : True

```

Dai risultati mostrati precedentemente si evince che la similarità media si ferma al 66%, e quasi l'80% dei turni (2053 su 2568) vengono sostituiti col golden. Le due conversazioni si comportano in maniera identica tra loro, evidenziando un comportamento costante del modello su tutto il dataset. Le coppie infatti, differiscono per dettagli minimi, come ad esempio una virgola messa diversamente, un apostrofo, una consonante raddoppiata o meno. Per questi casi specifici, è stata mantenuta la frase generata dall'LLM dal momento che il napoletano, non avendo un'ortografia standard, può essere scritto in modi leggermente diversi restando comunque corretto.

Un altro caso individuato riguarda un turno il cui testo originale era semplicemente «xxxxx», simbolo usato dal corpus per il parlato incomprensibile. Il golden standard riporta correttamente lo stesso segnaposto, mentre Gemma ha generato il testo «Ti sei dimenticato di inserire la frase da tradurre, scrivimi cosa vuoi che traduca», interpretando la sequenza di x come un prompt vuoto anziché come testo da riprodurre. In questo caso è stato mantenuto il testo originale riportato nel golden standard.

Altra casistica individuata riguarda la presenza di alcune frasi in inglese presenti nella conversazione, in particolare in sei turni specifici la cui "fonte italiana" non era in realtà italiana ma inglese (frasi tipo "why" o "never stop dreaming"). Per questi casi specifici è stata effettuata la traduzione in italiano e, successivamente, la traduzione in napoletano. Sono presenti però anche dei casi in cui alcune frasi in inglese sono state mantenute nella loro forma originaria. In particolare, è stato individuato un turno che contiene nella frase "fish and chips" che, in questo caso, è stato utilizzato per indicare il nome di un piatto.

1.4 Dataset Finale

In seguito alle modifiche effettuate e descritte nel paragrafo precedente, è stato costruito il nuovo dataset che presenta la stessa struttura di quello precedente, ma con l'aggiunta delle colonne **napoletano**, **fonte** e **similarità** dove:

- **napoletano**: rappresenta la frase corrispondente in italiano tradotta in napoletano.
- **fonte**: in seguito alla traduzione delle frasi in napoletano tramite l'utilizzo del modello gemma4 e successivamente verificate tramite golden-standard, tale sezione presenta due valori: **golden** e **gemma4** che, in base all'indice di similarità delle due traduzioni, se risulta minore dell'85%, allora la traduzione di **gemma4** viene sostituita da quella del **golden-standard**, altrimenti in caso contrario, viene mantenuta la traduzione effettuata da **gemma4**.
- **similarità**: rappresentata in percentuale, indica la similarità tra le frasi in napoletano tradotte tramite il modello **gemma4** e le frasi in napoletano del **golden-standard**.

2 Fine-Tuning attraverso l'utilizzo del modello Minerva

A partire dal dataset finale descritto nella sezione precedente, il chatbot è stato addestrato specializzando Minerva-7B-instruct-v1.0 [4], un modello linguistico italiano di 7 miliardi di parametri, tramite tecniche di *parameter-efficient fine-tuning*. Anziché aggiornare l'intero insieme dei pesi del modello, si è adottato l'approccio QLoRA (*Quantized Low-Rank Adaptation*): il modello di base viene caricato in precisione ridotta a 4 bit e mantenuto congelato, mentre l'addestramento aggiorna esclusivamente un piccolo insieme di matrici a basso rango (*adapter* LoRA) inserite nei moduli di proiezione dell'attenzione e del blocco *feed-forward*. Questa scelta consente di adattare un modello da 7 miliardi di parametri su una singola GPU Tesla T4 (15,6 GB di VRAM, ambiente Kaggle), mantenendo comunque una capacità di adattamento sufficiente a un compito linguistico specifico come il napoletano.

Il compito complessivo, ovvero traduzione dall'italiano al napoletano, completamento di dialogo e generazione libera di una risposta coerente, è stato scomposto in tre task distinti, ciascuno dei quali verranno trattati singolarmente nelle sezioni successive. I tre task condividono tuttavia la stessa pipeline di addestramento a monte, articolata in tre step progressivi, eseguiti nell'ordine seguente:

- **Step 1: Continued Pretraining sul napoletano** Il modello di base conosce l'italiano molto bene ma il napoletano solo marginalmente: addestrarlo direttamente sulle coppie prompt-risposta di un singolo task rischia di insegnargli a condizionare correttamente solo i primi token della risposta, lasciando inalterata la distribuzione linguistica di fondo, che resta italiana. Per questo, prima di affrontare qualunque task, un primo adapter LoRA viene addestrato come puro *language modeling* su tutto il testo napoletano disponibile nei soli file di training dei tre layout (mai su sviluppo o test, per evitare fughe di informazione verso la valutazione), spostando la distribuzione interna del modello verso il dialetto invece di limitarsi a condizionarne l'innescio.
- **Step 2: Iniezione lessicale** Il continued pretraining da solo lascia fragile il vocabolario dialettale specifico, il cui segnale è diluito su turni interi di molte parole in gran parte condivise con l'italiano. Un secondo adapter, che riparte da quello dello step 1, viene quindi specializzato su alcune migliaia di esempi corti e mirati, come glosse italiano-napoletano, esercizi di *cloze* in cui va prodotta la singola parola dialettale mancante nel suo contesto conversazionale, turni brevi ad alta densità dialettale in questo modo il gradiente si concentra esattamente dove il dialetto si gioca. Questo Step è eseguito una sola volta per modello e il suo adapter costituisce il punto di partenza condiviso da tutti e tre i task successivi.
- **Step 3: Fine-Tuning del task specifico** A partire dall'adapter dello Step 2, ciascun task (T1, T2, T3) prosegue l'addestramento in modo indipendente sui propri soli esempi, con iperparametri scelti appositamente (lunghezza massima della sequenza, numero di epoche, *learning rate*, metrica di selezione del checkpoint), dando origine a tre adapter finali distinti.

Quest'architettura a step condivisi consente di eseguire gli step 1 e 2 una sola volta per ciascun modello, invece di ripeterli per tutti e tre i task, riducendo così il costo computazionale complessivo. Le fasi comuni vengono descritte una sola volta, mentre nella sottosezione dedicata a ciascun task rientrano soltanto le scelte specifiche di quel task, per una trattazione più ordinata e priva di ripetizioni.

2.1 Definizione del task relativo alla Traduzione

Il task T1 «Traduzione con contesto» chiede al modello di tradurre in napoletano una singola frase italiana, fornendo come contesto anche i tre turni di conversazione immediatamente precedenti, già espressi in napoletano. Il formato del prompt è identico, byte per byte, a quello condiviso dagli altri due task, così che il modello impari un'unica convenzione di lettura del contesto conversazionale indipendentemente dal compito richiesto:

Conversazione finora:

A: emh

Traduci in napoletano: tu non hai mai fatto queste foto quando sei andata in barca

a cui corrisponde il target atteso «tu nun l'hê maje fatte sti ffoto quanno si' ghiuta 'ncopp'â varca». Fornire il contesto conversazionale, anziché trattare la frase come isolata, nasce dall'osservare che il parlato spontaneo raccolto nel corpus è frequentemente frammentario e ellittico: la forma corretta di una frase (un pronome, un tempo verbale, una scelta lessicale) dipende spesso da ciò che si sta discutendo, non solo dalla frase stessa.

Rispetto agli altri due task, il task T1 ha una proprietà che ne semplifica la valutazione: a ogni frase italiana corrisponde un'unica traduzione di riferimento, per cui una metrica di sovrapposizione superficiale come il chrF costituisce un criterio di selezione legittimo, cosa non altrettanto vera per T2 e T3, dove più continuazioni sono ugualmente plausibili. Per questo motivo lo step 3 di T1 è stato configurato con una lunghezza massima di sequenza pari a 512 token, condizione necessaria poichè il prompt con tre turni di contesto è il più lungo tra i tre layout, e una soglia inferiore troncherebbe i target, che la funzione di perdita tratterebbe comunque come corretti, 4 epoche di addestramento, *learning rate* pari a 1×10^{-4} e selezione del checkpoint migliore in base al chrF calcolato sul set di sviluppo. Il training è stato condotto su 1134 esempi di addestramento e 270 di sviluppo (di cui 200 effettivamente usati nelle valutazioni periodiche per contenere i tempi), per un totale di 284 passi di ottimizzazione in circa 63 minuti.

2.2 Continued Pre-Training sul Napoletano

Prima ancora di insegnare al modello a svolgere uno qualsiasi dei tre task, è stato necessario affrontare un problema a monte: Minerva-7B-instruct, come la maggior parte dei modelli linguistici italiani, ha visto nel proprio addestramento originale una quantità di testo napoletano pressoché trascurabile. Conosce l'italiano molto bene, il napoletano quasi per nulla. Se l'addestramento fosse partito direttamente dai task supervisionati (coppie prompt italiano $\rightarrow$ risposta napoletana), l'adapter LoRA avrebbe imparato soprattutto a *innescare* la risposta corretta nei primi token, sfruttando la regolarità dell'istruzione che vede sempre uguale ma la distribuzione di probabilità di fondo del modello, cioè ciò che gli «viene naturale» produrre quando non è vincolato token per token dalla supervisione, sarebbe rimasta quella di un modello che pensa in italiano. L'effetto è stato osservato concretamente nei log delle prime prove di addestramento: generazioni che iniziano correttamente in napoletano e che, allontanandosi dal prompt, scivolano progressivamente in italiano, come nell'esempio «però poi ce sta sempre l'incognita che può essere un veleno per la», interrotto proprio nel momento del passaggio di lingua.

Lo step 1 affronta il problema alla radice, con un addestramento diverso nella sua natura rispetto a quello dei task: non più coppie prompt-risposta con una parte della sequenza mascherata, ma puro *language modeling*. Tutto il testo napoletano disponibile viene ricostruito a partire dai turni annotati nei tre layout T1/T2/T3 non solo le porzioni usate come *target* nei task, ma il turno intero e passato al modello riga per riga, calcolando la funzione di perdita su ogni singolo token della sequenza, senza alcuna parte nascosta. La ricostruzione dei turni interi richiede un piccolo accorgimento, perché ciascun layout espone il turno in modo diverso: in T1 e T3 il *target* coincide con il turno intero, mentre in T2 il turno è spezzato fra il prefisso contenuto nel prompt («Continua il turno in napoletano:

...») e il target che ne rappresenta il completamento; per ogni coppia (conversazione, indice di turno) viene quindi tenuta la ricostruzione più lunga fra quelle disponibili nei tre layout. L'obiettivo di questo Step è spostare realmente la distribuzione interna del modello verso il napoletano, non limitarsi a condizionarne l'innescio iniziale — ed è anche per questo che il volume di testo impiegato (circa 11.200 parole, 21.500 token) supera quello usato come sola supervisione in uno dei tre task.

Per correttezza metodologica, questo testo viene ricostruito esclusivamente a partire dai file di *train* dei tre layout, mai da quelli di sviluppo o di test: se anche solo il testo (senza la struttura prompt-risposta) dei turni di valutazione finisse nel pretraining, il modello avrebbe già «letto» il materiale su cui verrà poi giudicato, il che costituirebbe una forma di *leakage* verso la valutazione finale. Il set di sviluppo viene comunque utilizzato, separatamente, per calcolare la *perplexity* di monitoraggio durante l'addestramento.

Nel run condotto su Minerva-7B-instruct, lo step 1 ha coinvolto 1147 turni di addestramento (11.229 parole, 21.513 token, impacchettati in 42 blocchi da 512 token) e 271 turni di sviluppo, con un adapter LoRA di rango 16 ($\alpha = 32$, *dropout* 0,05) addestrato per 3 epoche (18 passi totali) con *learning rate* 5×10^{-5} . La *perplexity* sul set di sviluppo è scesa da un valore iniziale di 19,72 a un minimo di 17,37, confermando che l'adattamento linguistico generale al napoletano ha effettivamente ridotto l'incertezza del modello su questo tipo di testo prima ancora di affrontare qualunque task specifico.

2.3 Iniezione Lessicale

Il secondo step intermedio nasce da un'osservazione pratica sui limiti del solo continued pretraining: anche dopo lo step 1, il vocabolario dialettale specifico (le singole parole, le forme verbali particolari, i costrutti tipici) risulta ancora incompleto, perché in un turno di dieci o quindici parole il segnale di perdita associato a ciascuna parola effettivamente dialettale è diluito fra tutte le altre parole del turno, la maggior parte delle quali condivisa con l'italiano. Lo step 2 prova a rinforzare proprio questo aspetto: riparte dall'adapter già addestrato nello step 1, continua ad addestrare, caricandolo con `is_trainable=True` anziché come modulo congelato e lo specializza su alcune migliaia di esempi molto più corti e mirati, generati appositamente in tre forme:

- una **glossa** nuda, che accosta direttamente una parola o breve espressione italiana alla sua resa napoletana, senza contesto conversazionale;
- un esercizio di **cloze in contesto**, in cui il modello deve produrre la singola parola dialettale mancante dentro una frase e dentro i turni di conversazione che la precedono, ad esempio, dato il contesto «PKP013: no, comunque ce sta / PKP013: vedimmo, magari è na bestia 'e satana stu criaturo» e la frase parzialmente tradotta «ma no, dai, secondo me ___ tene ll'età cchiù», il target atteso è «diec'anne,»;
- un **turno breve** ad alta concentrazione di forme dialettali, estratto così com'è dalla conversazione.

Con target di una sola parola o di poche parole, il gradiente si concentra esattamente dove il dialetto si gioca, invece di perdersi nella media di un turno intero. Il training set

costruito per questo Step è composto da 1742 esempi (1186 cloze, 384 turni brevi, 172 glosse) e il set di sviluppo da 151 esempi; l'adapter, sempre di rango 16, è stato addestrato per 3 epoche (327 passi, *learning rate* 5×10^{-5} , penalità di ripetizione disattivata poiché dannosa su target di una sola parola), raggiungendo un chrF di selezione pari a 68,95 al passo 252. Il *learning rate* contenuto e il numero limitato di epoche non sono una scelta arbitraria: con circa 4.000 esempi costruiti su un lessico di solo alcune centinaia di forme dialettali distinte, ogni forma viene vista molte volte, e spingere oltre l'addestramento rischia di far degenerare l'apprendimento in una semplice memorizzazione delle coppie (italiano, napoletano) come tabella di corrispondenza, a scapito della fluenza nei task successivi.

Questo step viene eseguito una sola volta per ciascun modello di base, non una volta per task: l'adapter che ne risulta costituisce il punto di partenza condiviso per tutti e tre i fine-tuning specifici (T1, T2, T3) descritti nello step 3, così il progetto esegue tre run di specializzazione invece di nove run completamente indipendenti. Il criterio di validazione previsto per questo Step è che il richiamo lessicale dialettale misurato sui tre task, a valle dello step 3, risulti superiore a quello di un identico fine-tuning fatto ripartire dal solo step 1: se così non fosse, il problema non starebbe nell'iniezione lessicale in sé ma nel formato dei prompt con cui viene veicolata.

2.4 Fine-Tuning per Traduzione

Questa fase non introduce soltanto la nuova configurazione di iperparametri già descritta nel paragrafo §2.1, ma presenta la stessa infrastruttura di addestramento condivisa con l'iniezione lessicale dello Step 2 e con gli altri due task (T2 e T3), in modo tale che in caso di divario di punteggio tra T1 e gli altri task, esso dipende dal task in sé e non dall'implementazione differente tra un run e l'altro. L'addestramento riparte dall'adapter dello step 2 (§2.3): non ne crea uno nuovo, lo carica con `is_trainable=True` e continua ad aggiornarlo. LoRA viene applicato agli stessi 224 moduli di proiezione selezionati per nome completo e non per semplice suffisso, mentre il modello di base resta quantizzato a 4 bit (QLoRA, `nf4` con doppia quantizzazione) e congelato.

Come per gli altri due step, ogni esempio viene tokenizzato due volte (una volta il solo prompt, una volta prompt più *target*) e i token corrispondenti al prompt vengono esclusi dal calcolo della funzione di perdita (-100 sull'etichetta), così che il modello si alleni esclusivamente a produrre la traduzione attesa e non a riprodurre l'istruzione. Un controllo di sanità verifica che il prompt tokenizzato sia effettivamente un prefisso esatto della sequenza completa, per accorgersi di eventuali disallineamenti introdotti dal *chat template* del tokenizer ed evitare in questo modo di corrompere silenziosamente la loss; sul run di T1 questo controllo, così come quello sul troncamento dei target oltre i 512 token, non ha rilevato alcuna anomalia, né in *train* né in *dev*.

La cadenza delle valutazioni periodiche non è un valore fisso ma viene derivata dagli step per epoca: con un *batch* di 16 esempi si ottengono 71 passi per epoca, da cui una valutazione ogni 35 passi e un totale di 8 valutazioni lungo le quattro

epoche (9 passi di *warm-up* iniziali), sufficienti per farsi che l'*early stopping* si attivi. Poiché l'ambiente di calcolo (Kaggle) azzerava lo spazio di lavoro fra una sessione e l'altra, l'addestramento prevede inoltre la possibilità di riprendere l'addestramento dalla sessione precedente, copiando i checkpoint nello spazio scrivibile della sessione corrente.

A ogni valutazione vengono calcolate alcune metriche, ovvero chrF, BLEU, un F1 lessicale a *bag-of-words*, l'accuratezza a livello di token in modalità *teacher forcing*: il modello riceve il prompt e si osserva, posizione per posizione, quale token è previsto con probabilità più alta, confrontandolo con quello corretto di riferimento, senza mai generare in autonomia. Questa modalità è economica ma strutturalmente cieca di fronte ad un problema di generazione libera: un modello può ottenere un punteggio dignitoso in *teacher forcing* ma allontanarsi dalla risposta attesa o cadere in ripetizioni dal momento che, la generazione libera consiste nel generare token dopo token senza conoscere la risposta corretta a ogni passo. Per questo motivo, oltre a queste metriche, viene eseguita anche una generazione autoregressiva reale su un sottoinsieme del set di sviluppo, in modo da disporre già durante l'addestramento di un segnale meno ingannevole.

Al termine dei 284 passi di ottimizzazione, il checkpoint selezionato come migliore secondo il chrF di monitoraggio è quello al passo 245, e costituisce l'adapter finale di T1. Sulla base di quanto esposto fino ad ora, sono stati analizzati in seguito, nel dettaglio, i risultati ottenuti per questo task di riferimento.

2.5 Analisi dei risultati

La Figura 1 sottostante riassume l'andamento della funzione di perdita nei tre step che portano all'adapter finale T1. Nello Step di *continued pretraining* dialettale (a sinistra) la perdita di validazione scende in modo monotono da circa 2,98 a un minimo di 2,855, senza mai divergere dalla perdita di addestramento: il modello sta assorbendo la distribuzione linguistica del napoletano senza segni di sovradattamento. Nello Step di iniezione lessicale (al centro) la perdita di addestramento crolla da oltre 4 a circa 0,35, mentre quella di validazione si stabilizza intorno a 0,749; la forbice che si apre fra le due curve è attesa e benigna, perché lo Step è deliberatamente costruito su esempi corti e ripetitivi in cui il modello memorizza il lessico dialettale mirato. Nello Step di traduzione vero e proprio (a destra) la perdita di validazione raggiunge il minimo di 0,514 intorno all'epoca 2,5 e da lì risale leggermente, mentre la perdita di addestramento continua a scendere fino a circa 0,07: è esattamente il punto oltre il quale il modello smetterebbe di generalizzare per iniziare a memorizzare, e non a caso il checkpoint finale è stato selezionato in prossimità di questo minimo, al passo 245.

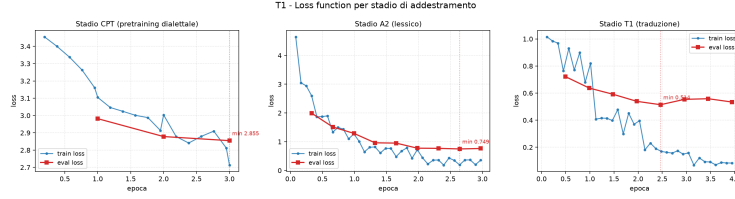


Figura 1: Grafico della Loss-Function per il Fine-Tuning sulla Traduzione in dialetto Napoletano

La Figura 2 successiva invece mostra l'evoluzione delle metriche di monitoraggio calcolate in *teacher forcing* sul set di sviluppo lungo le quattro epoche. Precision, Recall e F1 basati sulla sovrapposizione *bag-of-words* (a sinistra) crescono di pari passo da circa 0,65 a circa 0,76, con la Recall costantemente un soffio sopra la Precision: il modello recupera quasi tutte le parole di riferimento senza inflazionare l'output con termini spuri. Il pannello di destra conferma la stessa tendenza sulle metriche di somiglianza superficiale: il chrF++ sale da 66,2 a poco meno di 76 e il BLEU da 39,2 a circa 55,7, entrambi con un chiaro plateau fra l'epoca 3,5 e la 4, coerente con la scelta di fermare l'addestramento a quattro epoche.

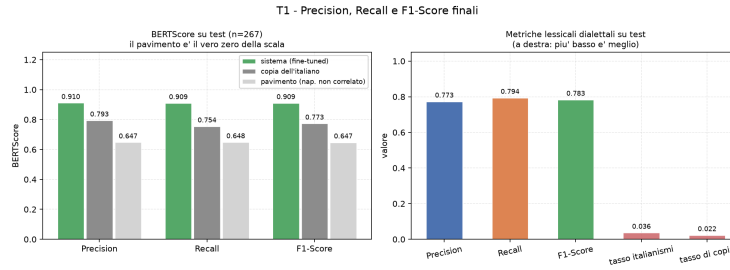


Figura 2: Grafico di confronto metriche Precision, Recall e F1-Score per il Fine-Tuning sulla Traduzione in dialetto Napoletano

Infine, le Figure 3 e 4 riportano la valutazione finale sul set di test, questa volta in generazione autoregressiva reale e non più in *teacher forcing*. Sul chrF++ (Figura 4) il sistema *fine-tuned* raggiunge 71,05, con intervallo di confidenza al 95% via bootstrap che non si sovrappone ad alcuna baseline: il divario rispetto alla semplice copia dell'italiano (35,41) è di oltre 35 punti, e rispetto al riferimento di un altro item, cioè il pavimento della scala che misura le somiglianze puramente casuali (9,62), è di oltre 61 punti. Il modello non sta quindi né limitandosi a ricopiare l'input italiano né producendo napoletano generico scollegato dalla frase. La Figura 2 rafforza questa lettura da due angolazioni. A sinistra, il BERTScore sul test (n=267) attribuisce al sistema una F1 di 0,909, contro 0,773 della copia dell'italiano e 0,647 del pavimento (napoletano non correlato): posizionando quest'ultimo come vero zero della scala, il sistema copre quasi interamente la distanza che separa una traduzione casuale da quella di riferimento. A destra, le metriche lessicali specificamente dialettali confermano che il modello ha appreso il registro e non soltanto il significato: Precision, Recall e F1 sul lessico napoletano si attestano rispettivamente a 0,773, 0,794 e 0,783, mentre il tasso di italianismi residui è appena 0,036 e il tasso di copia dall'italiano 0,022, entrambi valori molto bassi in cui, trattandosi di indicatori di errore, il basso è meglio. Nel complesso, il

task di traduzione con contesto è quello in cui il sistema fornisce la prova più netta di aver interiorizzato il dialetto.

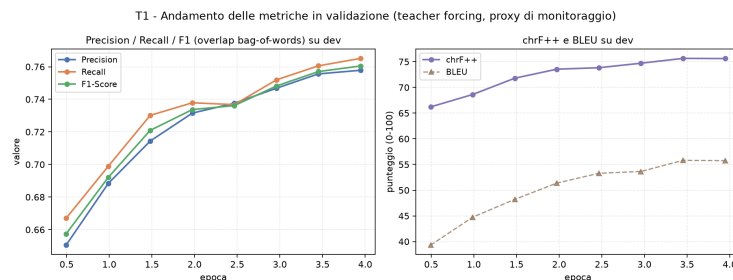


Figura 3: Grafico andamento delle metriche Precision, Recall e F1-Score per il Fine-Tuning sulla Traduzione in dialetto Napoletano

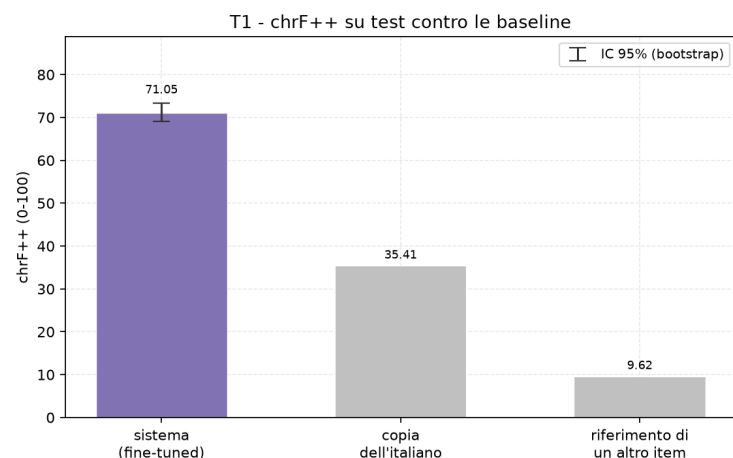


Figura 4: Grafico del chrF per il Fine-Tuning sulla Traduzione in dialetto Napoletano

2.6 Fine-Tuning per Completamento di dialogo

Il task T2 «Completamento di dialogo» chiede al modello di portare a termine un turno di parola già avviato, in modo coerente con la conversazione che lo precede. Rispetto a T1 cambia la natura stessa di ciò che viene richiesto: il modello non deve più rendere in napoletano un contenuto che gli è stato fornito, ma decidere autonomamente come deve proseguire un dialogo di cui conosce soltanto la prima parte iniziale. Il prompt condivide con gli altri due task la stessa intestazione e gli stessi tre turni di contesto, e si chiude con il prefisso del turno da completare:

Conversazione finora:

A: emh

Continua il turno di B: tu nun l'he maje fatte sti ffoto

a cui corrisponde il target atteso «quando si' ghiuta 'ncopp'a varca». Il punto di taglio non è fisso ma proporzionale: il turno viene spezzato in media al 49,8% della sua lunghezza, così che il modello non possa imparare a riconoscere una posizione ricorrente e sia costretto a dedurre dal prefisso quanto ancora manca. I target che ne risultano sono brevi, in media

5,23 parole e mediana 4 sul set di addestramento (5,09 e 4 su sviluppo, 4,99 e 4 su test) e questo ha una conseguenza diretta sulle metriche: su stringhe così corte una singola parola sbagliata sposta il punteggio di sovrapposizione di molti punti.

Una scelta apparentemente minore ha in realtà un peso considerevole: l’istruzione dichiara esplicitamente *chi* sta parlando («Continua il turno di B»). Senza questa indicazione, di fronte a un contesto come «A: emh» il modello non avrebbe modo di sapere se debba replicare ad A o proseguire il turno di A stesso, e il compito diventerebbe ambiguo per costruzione; nel corpus l’ultimo turno di contesto appartiene allo stesso parlante del turno da completare in 1214 casi su 3400. Per la stessa ragione il prompt viene reso nello stile *prefill*: il prefisso non viene ripetuto in coda all’istruzione, ma collocato all’interno del messaggio dell’assistente, così che il modello prosegua con la generazione del proprio testo invece di ricollegarsi ad un frammento mostrato nelle fasi precedenti.

Anche il calcolo della funzione di perdita si discosta da quello di T1. In T1 i token del prompt sono esclusi del tutto, con etichetta -100, perché l’istruzione e la frase italiana non fanno parte di ciò che il modello deve produrre. In questo caso specifico, invece, il prefisso è parte integrante dello stesso turno del target: è napoletano, è stato pronunciato dal parlante di cui si sta completando l’enunciato, e ignorarlo del tutto significherebbe gettare via metà del segnale dialettale disponibile. Il prefisso riceve perciò un peso ridotto ma non nullo, pari a 0,3, mentre l’intestazione e il contesto restano completamente mascherati e il target pesa 1: in questo modo il modello apprende il registro dell’intero turno senza che la parte già data domini il gradiente.

I dati impiegati sono 3400 esempi di addestramento (3154 provenienti dal golden standard e 246 dalla traduzione automatica validata), 103 di sviluppo e 106 di test; il contesto conta tre turni in 3393 casi su 3400, e i pochi item con contesto più breve sono quelli di inizio conversazione. Lo split non è cronologico, ma una zona cuscinetto fra le partizioni garantisce che nessun item abbia un contesto a cavallo del confine fra addestramento, sviluppo e test: senza quell’accorgimento un turno di test potrebbe comparire come contesto di un turno di addestramento, e la valutazione ne risulterebbe inquinata. Il lessico dialettale osservato su questo task conta 258 tipi distinti.

La configurazione dell’addestramento tiene conto di due fattori: la maggiore quantità di esempi disponibili per questo task e la brevità dei suoi target. Il *learning rate* è stato fissato a 2×10^{-5} , cinque volte più basso di quello di T1: con 3400 esempi ogni epoca corrisponde a 212 passi di ottimizzazione, e un passo troppo ampio finirebbe per far oscillare la perdita. Sono previste 6 epoche, con lunghezza massima di sequenza pari a 512 token e un *batch* di 16 esempi, ottenuto elaborando 2 esempi per passo e accumulando il gradiente su 8 passi. L’adapter LoRA è di rango 8 ($\alpha = 16$, *dropout* 0,1), affiancato da un *weight decay* di 0,01 e da NEFTune con $\alpha = 5$.

La selezione del checkpoint avviene sulla *eval loss*, con *patience* pari a 4. Prima dell’addestramento sono state fissate le baseline di riferimento per la lettura dei risultati. Sul set di sviluppo il pavimento ottenuto mescolando i target vale 8,99 di chrF++, la baseline che

ripete il prefisso vale 13,80 e quella che ripete l'ultimo turno di contesto 7,41; su test i tre valori sono rispettivamente 11,38, 13,61 e 10,00. La distanza fra il pavimento e la baseline che ripete il prefisso corrisponde all'intervallo utile della metrica: meno di cinque punti su sviluppo e poco più di due su test. La densità dialettale dei riferimenti vale 0,256 su sviluppo e 0,280 su test.

2.7 Analisi dei Risultati

La Figura 5 lo rende evidente già dalla funzione di perdita. La perdita di validazione raggiunge il minimo di 3,257 esattamente all'epoca 1,00 e da lì risale in modo monotono fino a 3,8, mentre la perdita di addestramento continua a scendere fino a circa 0,85: il classico profilo a forbice della memorizzazione. L'addestramento, previsto su 6 epoche, si è arrestato a 2,99 per *early stopping* con *patience* pari a 4, e il checkpoint è stato correttamente fissato al minimo della validazione. Superata la prima epoca, ogni ulteriore passo non insegna al modello a completare meglio, ma a ricordare i completamenti visti in addestramento.

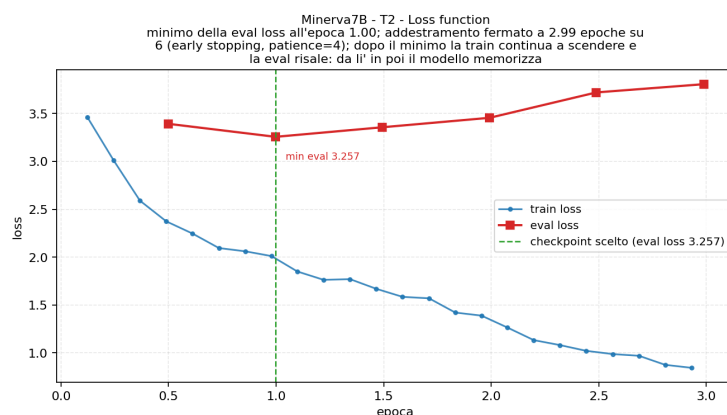


Figura 5: Grafico della Loss-Function per il Fine-Tuning sul Completamento di dialogo

Le Figure 6 e 7 adottano allora due metriche più adatte a un compito aperto, entrambe misurate sul solo target con contesto e prefisso veri in input. La `ctx_accuracy` (Figura 6) misura la quota di item in cui il modello assegna maggiore probabilità al target quando è preceduto dal contesto vero piuttosto che da un contesto estraneo: raggiunge il massimo di 0,719 al passo 212 (epoca 1), ben al di sopra sia del caso puro (0,5) sia della prestazione zero-shot su sviluppo (0,641), a conferma che il modello sta effettivamente usando il contesto e non ignorandolo. Dopo il passo 212 la metrica declina, ricadendo sotto il livello zero-shot: la memorizzazione osservata nella loss erode proprio la sensibilità al contesto. La perplexity crolla da 99,6 in zero-shot a un minimo di 25,29 al passo 212, per poi risalire progressivamente fino a circa 44. Entrambe le curve individuano nel passo 212 lo stesso punto ottimale, coerente con il minimo della loss di validazione.

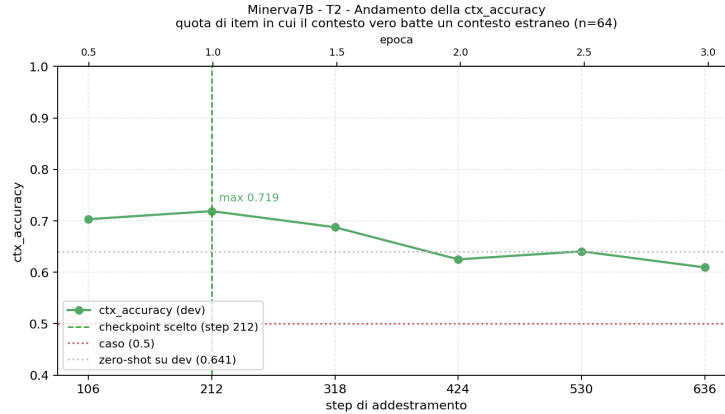


Figura 6: Grafico di `ctx_accuracy` per il Fine-Tuning sul Completamento di dialogo

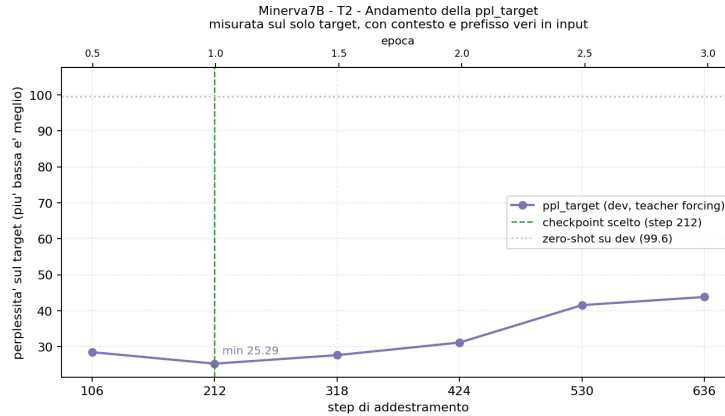


Figura 7: Grafico dell'andamento della perplexity per il Fine-Tuning sul Completamento di dialogo

La Figura 8 confronta su sviluppo (n=103) tre configurazioni: zero-shot, few-shot a 4 esempi e fine-tuned. In *teacher forcing* few-shot e zero-shot coincidono, poiché gli esempi entrano solo nel prompt di generazione: è per questo che le due colonne di perplexity sono identiche (99,6). Il fine-tuning riduce la perplexity sul target a 25,3, un miglioramento di quasi quattro volte, ed è l'evidenza più forte del pannello. Le accuratèzze crescono coerentemente: l'accuratèzza a livello di token sale da 0,281 a 0,364, la `ctx_accuracy` da 0,641 a 0,689. Sulla forma del testo generato, la densità dialettale aumenta da 0,246 a 0,423, superando la densità dei riferimenti (0,256), e il tasso di copia resta prossimo a zero. Il chrF++, per contro, scende da 10,68 (zero-shot) a 9,49 (fine-tuned) e resta sotto la baseline banale del «ripeti il prefisso» (13,80): è il segnale che, su questo task, il chrF non è una metrica affidabile, perché premia la sovrapposizione superficiale con un unico riferimento laddove le continuazioni valide sono molte, e l'intera scala utile è compressa in pochi punti. La lettura corretta è quindi che il fine-tuning migliora tutto ciò che misura la qualità dialettale e la comprensione del contesto (perplexity, accuratèzze, densità), mentre penalizza soltanto una metrica di somiglianza letterale che qui è strutturalmente poco informativa.

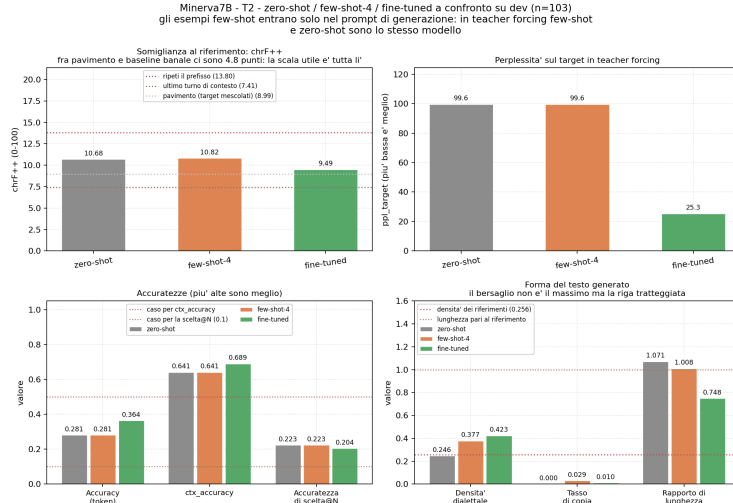


Figura 8: Grafico che mostra la differenza tra approccio zero-shot, few-shot e fine-tuned per il training per il Fine-Tuning sul Completamento di dialogo

Infine, la Figura 9 conferma il quadro sul set di test (n=106), dove il confronto si riduce a zero-shot contro fine-tuned. La perplexity sul target scende da 77,3 a 19,3, l'accuratezza a livello di token sale da 0,295 a 0,422, la *ctx_accuracy* da 0,613 a 0,660 e la densità dialettale da 0,326 a 0,453. Anche in questo caso il chrF++ arretra lievemente, da 11,71 a 10,32, e il rapporto di lunghezza scende da 0,998 a 0,750, indicando che il modello *fine-tuned* tende a produrre completamenti più brevi del riferimento. Nel complesso, sul completamento di dialogo il fine-tuning produce un modello che pensa in napoletano e sfrutta il contesto in modo misurabile, ma il cui merito non è catturato dalle metriche di sovrapposizione con un singolo riferimento.

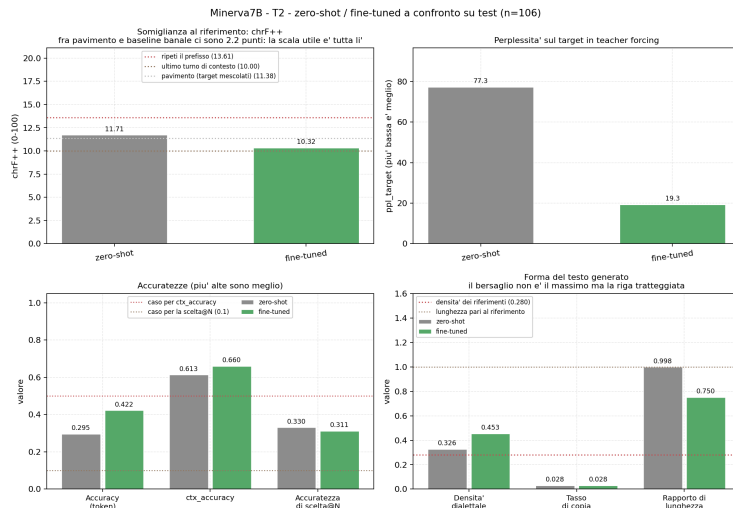


Figura 9: Grafico che mostra la differenza tra approccio zero-shot, few-shot e fine-tuned per il testing per il Fine-Tuning sul Completamento di dialogo

2.8 Fine-Tuning per Generazione Libera

Il task T3 «Generazione libera» rimuove l'ultimo vincolo rimasto: al modello vengono forniti soltanto i turni di contesto e l'indicazione di chi deve parlare, e da lì deve produrre per intero il turno successivo, decidendo autonomamente sia il contenuto sia la forma. L'istruzione non è una stringa fissa, ma varia fra alcune formulazioni equivalenti («Tocca a B. Rispondi in napoletano, restando sul tema.», «Continua la conversazione: scrivi il turno di B in napoletano.», «Turno di B. Rispondi in napoletano, breve e pertinente.»), che oltre alla lettera del parlante differiscono nella formulazione: se l'istruzione fosse sempre identica, il modello potrebbe agganciarsi a quella stringa e la valutazione non distinguerebbe più fra l'aver compreso il compito e l'aver memorizzato il preambolo.

Questo task pone un problema che gli altri due non presentano, ed è il motivo per cui la sua configurazione si discosta dalle precedenti. La *cross-entropy* sui circa 750 target disponibili non è in grado, da sola, di insegnare la pertinenza conversazionale: dato un contesto, le repliche valide sono centinaia, mentre il gradiente comunica al modello «produci esattamente questa stringa». Il modello può quindi minimizzare la perdita imparando la forma superficiale dei turni napoletani senza che il contesto incida su ciò che dice, ovvero il fenomeno della cecità al contesto osservato in fase di valutazione su tutti e tre i modelli.

La contromisura adottata è un termine contrastivo aggiunto alla funzione di perdita. Ogni esempio viene elaborato due volte, una con il proprio contesto reale e una con un contesto prelevato da un altro punto del corpus, e alla *cross-entropy* si somma il termine

$$\lambda \cdot \max(0, m - [\log P(\text{target} \mid \text{ctx vero}) - \log P(\text{target} \mid \text{ctx falso})])$$

con $\lambda = 0,5$ e margine $m = 0,15$ nat per token. La penalità si annulla non appena il target risulta più verosimile con il contesto vero che con quello falso di almeno il margine richiesto, altrimenti resta attiva: è il termine che rende il contesto *causalmente* rilevante per l'ottimizzazione. Questo non è un metodo di allineamento alle preferenze come DPO: non richiede né un modello di riferimento congelato né coppie di risposte annotate da un essere umano, poiché l'esempio negativo viene costruito ricombinando i dati già disponibili.

Per la stessa ragione cambia anche il criterio di selezione del checkpoint, che qui è il `ctx_delta`, cioè la media, su un campione di item, della differenza $\log P(\text{target} \mid \text{ctx vero}) - \log P(\text{target} \mid \text{ctx falso})$ misurata in nat per token: sopra zero il contesto viene sfruttato, a zero il modello è cieco. La selezione sul chrF, adottata su T1, premierebbe qui il checkpoint che indovina meglio le parole dell'unico turno realmente pronunciato, che non è l'obiettivo di questo task.

Una terza differenza riguarda il punto di partenza: T3 non riparte dall'adapter dello Step di iniezione lessicale ma dal modello di base. La pertinenza conversazionale è una proprietà che il modello di base già possiede, ed è proprio quella che il *continued pretraining* tende a erodere, spostando la distribuzione verso la forma dei turni dialettali; poiché su questo task la pertinenza è l'obiettivo primario, si è scelto di non anteporvi i due step, riservando

l'eventuale rafforzamento della dialettalità a un intervento successivo.

L'addestramento si basa su 742 esempi di training, 165 di sviluppo e 168 di test, con 4 epoche previste, *learning rate* 1×10^{-4} , lunghezza massima di sequenza pari a 384 token (il layout non contiene il prefisso del turno ed è quindi il più corto dei tre), *batch* efficace di 16 esempi, adapter LoRA di rango 16 ($\alpha = 32$, *dropout* 0,05), due valutazioni per epoca e *patience* pari a 4. Il *ctx_delta* di monitoraggio viene calcolato su 64 item, con un costo di due soli passaggi in avanti per item e nessuna generazione. Il run complessivo è durato 108 minuti e il checkpoint promosso è quello del passo 161. Anche l'insieme dei token di fine turno viene verificato prima dell'avvio e non a posteriori, per evitare che una generazione troncata o vuota venga scambiata per una risposta di bassa qualità.

In seguito, verranno analizzati nel dettaglio i risultati ottenuti da questa configurazione.

2.8.1 Analisi dei Risultati

La Figura 10 rende esplicita questa scelta. La perdita di validazione tocca il proprio minimo di 3,099 già intorno all'epoca 2, resta pressoché piatta fino a quel punto e poi risale verso 3,73, mentre la perdita di addestramento prosegue la discesa fino a circa 0,95: sarebbe il momento canonico dell'*early stopping* sulla loss. Il checkpoint finale, però, è stato deliberatamente selezionato più avanti, all'epoca 3,43, in corrispondenza del massimo di *ctx_delta* (pari a +0,5252), cioè del punto in cui il modello mostra la maggiore differenza di verosimiglianza fra il target preceduto dal contesto vero e lo stesso target preceduto da un contesto estraneo. In altre parole, per la generazione libera si è data maggiore priorità al contesto conversazionale rispetto alla pura minimizzazione della perdita: si accetta una perdita di validazione più alta pur di scegliere il checkpoint che meglio lega la risposta generata alla conversazione in corso, che è l'obiettivo del task. La distanza fra le due linee verticali della figura (il minimo della loss a sinistra, il checkpoint scelto a destra) visualizza questo compromesso metodologico.

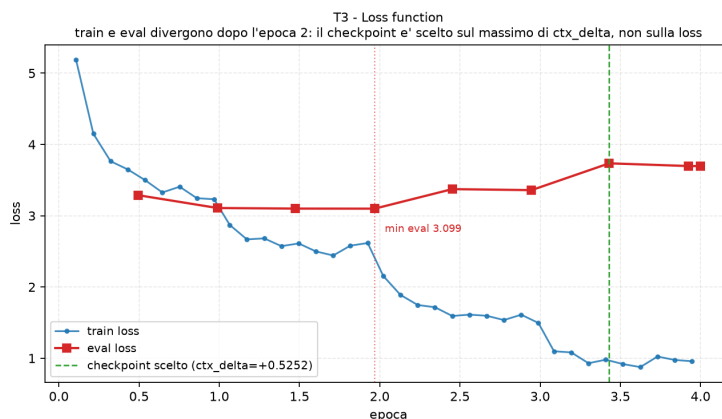


Figura 10: Grafico della Loss-Function per il Fine-Tuning sulla generazione libera

3 Fine-Tuning attraverso l'utilizzo del modello LLama

Il secondo modello di base impiegato è Llama-2-7b-chat-hf [5], un modello da 7 miliardi di parametri della stessa taglia di Minerva ma di natura multilingue: il suo corpus di addestramento copre numerose lingue, fra cui l'italiano, senza però essere specializzato su nessuna di esse in particolare. Mettere a confronto due modelli della stessa dimensione, uno specializzato sull'italiano e uno multilingue generalista, permette di distinguere quanto del risultato finale dipenda dall'adattamento al napoletano operato dal fine-tuning e quanto invece sia già contenuto nella competenza linguistica del modello di partenza.

L'infrastruttura di addestramento è quella già descritta nel paragrafo §2: stessi dati, stesso rendering dei prompt, stessa funzione di perdita, stesse metriche di monitoraggio e stessa procedura di selezione del checkpoint. L'unica differenza sostanziale riguarda la pipeline a monte: i due step intermedi di *continued pretraining* e di iniezione lessicale sono stati eseguiti soltanto per T1, mentre su T2 e su T3 l'addestramento riparte direttamente dal modello di base. Le sottosezioni che seguono si limitano quindi a commentare i risultati ottenuti sui tre task.

3.1 Analisi dei Risultati sulla Traduzione

Diversamente da quanto avviene su T2 e T3, il fine-tuning di T1 su Llama riparte dalla pipeline a tre step descritta nel paragrafo precedente: un primo adapter di *continued pre-training* sul napoletano, un secondo di iniezione lessicale e infine la specializzazione sul task di traduzione, che carica l'adapter dello Step precedente e continua ad aggiornarlo. Lo Step finale ha lavorato su 1134 esempi di addestramento e 270 di sviluppo (sottocampionati a 200 per le valutazioni periodiche), con *learning rate* 1×10^{-4} , 4 epoche, lunghezza massima di sequenza pari a 512 token e pesatura lessicale attiva, che triplica il peso dei 988 tipi dialettali riconosciuti e copre così il 50,2% dei token di *target*. Con un *batch* efficace di 16 esempi si ottengono 71 passi per epoca, 284 passi complessivi e 8 valutazioni intermedie.

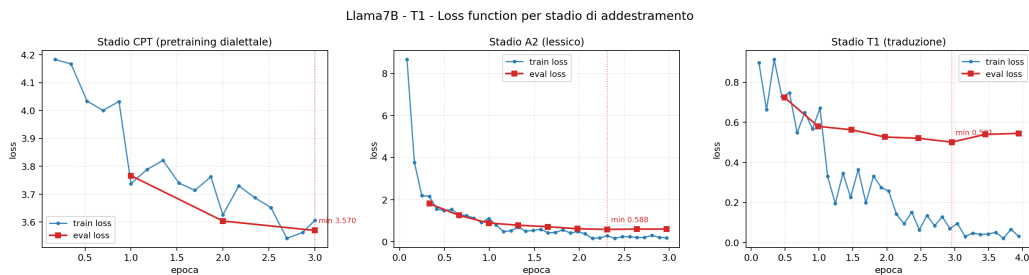


Figura 11: Grafico della Loss-Function per i tre step di addestramento di T1 con il modello Llama7B

La Figura 11 affianca i tre step in altrettanti pannelli, ciascuno con la propria scala verticale, dato che le grandezze in gioco differiscono di un ordine di grandezza; la linea verticale tratteggiata segna in ogni pannello il punto in cui viene fissato il checkpoint.

Nel pannello del *continued pretraining* la *train loss* scende da 4,18 a circa 3,55 lungo tre epoche, ma lo fa con oscillazioni ampie e senza mai stabilizzarsi, il che è atteso

su appena 18 passi di ottimizzazione: il testo napoletano ricostruito dai soli file di addestramento ammonta a 1147 turni, 11.229 parole e 23.989 token, impacchettati in 46 blocchi da 512. La *eval loss*, misurata una volta per epoca, scende invece in modo monotono da 3,766 a 3,604 e infine al minimo di 3,570, che coincide con la fine dell’addestramento; in termini di perplessità sul set di sviluppo il passaggio corrisponde a una discesa da 43,22 a 35,51. Nessuna delle due curve mostra segni di risalita, il che indica che lo Step si è fermato per esaurimento delle epoche previste e non per sovradattamento.

Il pannello dell’iniezione lessicale ha la forma tipica di un addestramento su esempi molto corti: la *train loss* crolla da 8,65 a meno di 2 nella prima mezza epoca e prosegue poi verso 0,2, mentre la *eval loss* scende da 1,81 fino al minimo di 0,588 all’epoca 2,31 e da lì in avanti resta piatta intorno a 0,60. Le due curve si separano nettamente nella seconda metà, con la *train loss* che continua a scendere e quella di validazione ferma: su 1742 esempi costruiti attorno a poche centinaia di forme dialettali distinte, il modello arriva presto al punto in cui rivedere le stesse coppie non aggiunge informazione. Il checkpoint conservato è quello del passo 288 su 327, dove il chrF di monitoraggio raggiunge 68,65.

Il pannello dello Step di traduzione mostra la dinamica più articolata dei tre. La *train loss* parte da 0,89 — un valore già basso, perché il modello arriva qui con due step di adattamento alle spalle — oscilla in modo marcato per tutta la prima epoca fra 0,55 e 0,91, poi scende in scalini successivi fino a valori intorno a 0,03 alla quarta epoca. La *eval loss* segue un andamento molto più liscio: 0,723 alla prima valutazione, poi 0,580, 0,563, 0,527, 0,520 e il minimo di 0,501 all’epoca 2,96, dove viene fissato il checkpoint; le due valutazioni successive risalgono a 0,540 e 0,545. La distanza fra le due curve, che alla fine supera i quattro decimi, e la risalita della sola *eval loss* collocano il passaggio alla memorizzazione poco dopo la terza epoca. L’*early stopping*, configurato con *patience* pari a 4, non arriva a intervenire perché restano solo due valutazioni dopo il minimo, e l’addestramento completa i 284 passi previsti.

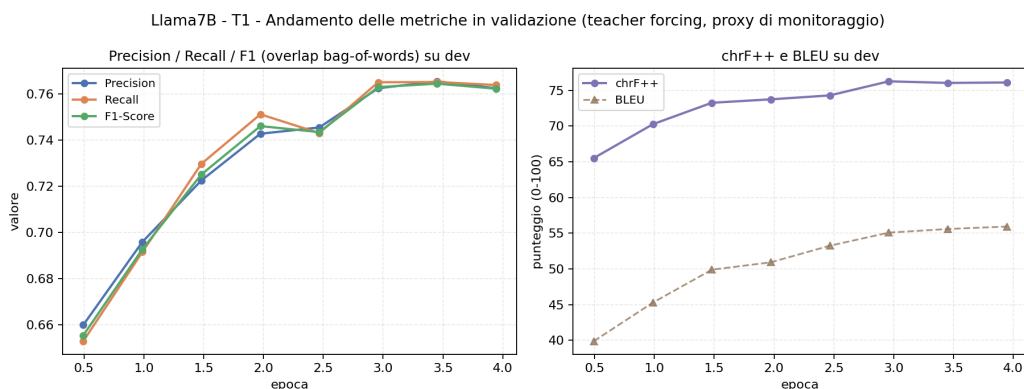


Figura 12: Grafico dell’andamento delle metriche in validazione durante il Fine-Tuning sulla Traduzione con contesto con il modello Llama7B

La Figura 12 riporta le metriche calcolate sul set di sviluppo lungo lo Step di traduzione, in due pannelli e su due scale diverse: a sinistra precision, recall e F1 lessicali a *bag-of-words*, a destra il chrF++ e il BLEU su scala da 0 a 100. Tutte sono misurate in *teacher forcing*, come ricorda il sottotitolo del grafico, e

vanno quindi lette come proxy di monitoraggio e non come stima della qualità in generazione.

Le tre curve del pannello di sinistra sono di fatto sovrapposte per tutta la durata dell'addestramento, con precision e recall che non divergono mai di più di un punto percentuale: il modello non è né sistematicamente prolioso né sistematicamente reticente nelle parole che sceglie. Partono da 0,660, 0,653 e 0,655 alla prima valutazione, salgono rapidamente fino a circa 0,725 all'epoca 1,5, presentano una lieve flessione all'epoca 2,47 e raggiungono il massimo di 0,762, 0,765 e 0,763 all'epoca 2,96, cioè esattamente al checkpoint selezionato. Le due valutazioni successive restano sullo stesso livello senza aggiungere nulla.

Il pannello di destra mostra la stessa forma con un dettaglio in più. Il chrF++ sale da 65,51 a 76,25 all'epoca 2,96 e poi si appiattisce su 76,03 e 76,09; il BLEU, invece, continua a crescere fino all'ultima valutazione, da 39,88 a 55,92, senza mai fermarsi. La divergenza fra le due curve nella parte finale è informativa: il BLEU premia la corrispondenza esatta di n-grammi interi, il chrF++ lavora sui caratteri e tollera le varianti ortografiche, per cui un modello che nelle ultime epoche impara a riprodurre alla lettera la grafia dei *target* di addestramento continua a guadagnare sul primo mentre ha già saturato il secondo. È lo stesso fenomeno che la *eval loss* registra come risalita, letto attraverso una metrica diversa.

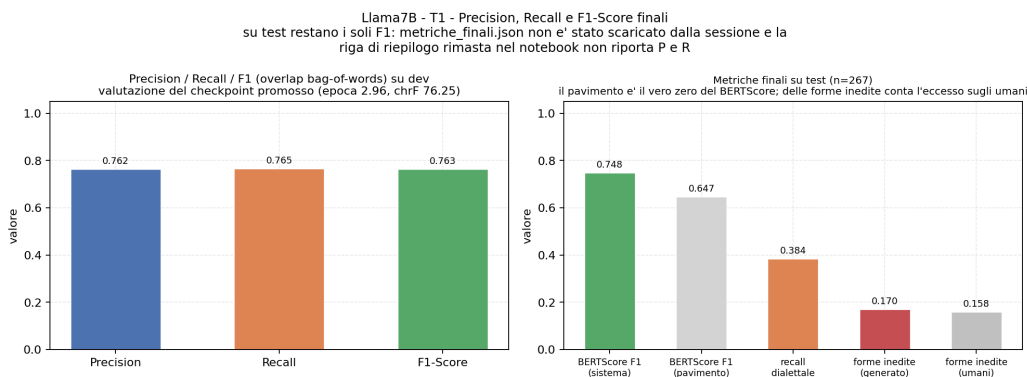


Figura 13: Grafico di confronto delle metriche Precision, Recall e F1-Score finali per il Fine-Tuning sulla Traduzione con contesto con il modello Llama7B

La Figura 13 riporta le metriche finali in due pannelli distinti, e il sottotitolo spiega perché i due non contengano le stesse quantità: il file con le metriche finali su test non è stato scaricato dalla sessione di calcolo e la riga di riepilogo rimasta nel notebook non riporta precision e recall, per cui su test sopravvivono soltanto le misure di tipo F1.

Il pannello di sinistra riguarda quindi il set di sviluppo e fotografa il checkpoint promosso, quello dell'epoca 2,96 con chrF 76,25: precision 0,762, recall 0,765 e F1 0,763, cioè i valori con cui si chiude la curva della figura precedente. Il pannello di destra riguarda invece il set di test, 267 esempi, e affianca quattro grandezze eterogenee, ciascuna con il proprio termine di paragone. Il BERTScore F1 del sistema vale 0,748 contro un pavimento di 0,647, dove il pavimento è il BERTScore calcolato fra riferimenti non

correlati fra loro ed è quindi il vero zero della scala, non lo 0,0 dell'asse: la distanza utile fra pavimento e sistema è di circa dieci punti percentuali su una scala che parte da 0,647. Il recall dialettale si ferma a 0,384, cioè il sistema recupera poco più di un terzo delle forme dialettali presenti nel riferimento. Le ultime due barre vanno lette insieme, come indica il sottotitolo: le forme inedite prodotte dal sistema valgono 0,170 contro lo 0,158 dei riferimenti umani, per cui il bersaglio non è massimizzare la barra ma avvicinarsi a quella umana, e l'eccesso osservato è di poco più di un punto percentuale.

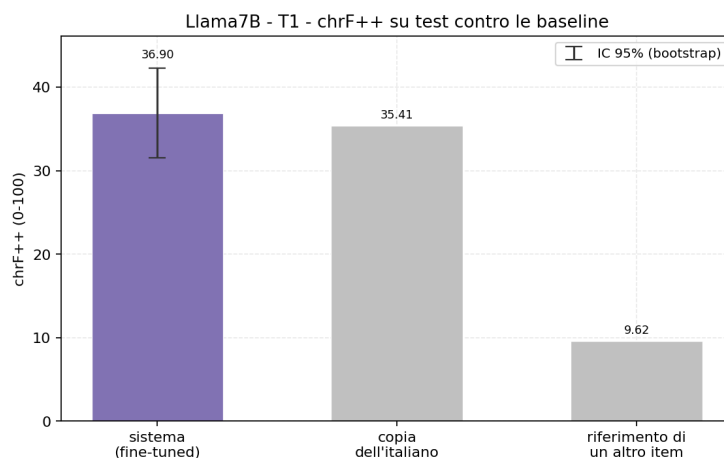


Figura 14: Grafico del chrF++ su test contro le baseline per il Fine-Tuning sulla Traduzione con contesto con il modello Llama7B

La Figura 14 mette il chrF++ del sistema fine-tunato accanto alle due baseline previste per questo task. La prima è la copia dell'italiano lasciato invariato, che su questo corpus è tutt'altro che banale perché molti turni coincidono nelle due lingue; la seconda è il riferimento di un altro item, cioè una traduzione corretta ma appartenente a un'altra frase, e serve a fissare il livello del caso.

Il sistema ottiene 36,90 con un intervallo bootstrap al 95% che va da 31,56 a 42,30, la copia dell'italiano ottiene 35,41 e il riferimento estraneo 9,62. La distanza dal livello del caso è quindi ampia e conferma che il modello produce traduzioni pertinenti all'item che gli viene dato; la distanza dalla copia dell'italiano è invece di appena 1,5 punti, e l'intervallo di confidenza del sistema contiene interamente il valore della baseline. Su questo campione di test, dunque, il grafico non consente di affermare che il sistema faccia meglio che lasciare la frase in italiano. Va notato che la baseline non ha barra di errore perché è un valore deterministico, mentre l'ampiezza dell'intervallo del sistema, quasi undici punti, riflette la dimensione ridotta del test e la forte variabilità dei punteggi da un item all'altro.

Il divario fra questa figura e le due precedenti è il risultato più importante del task, e ha una causa individuabile nei dati di valutazione. In *teacher forcing*, dove il modello riceve il *target* vero posizione per posizione, il chrF di monitoraggio arriva a 76,25; in generazione reale, dove deve produrre la frase da solo, scende a 36,90. La ragione è la lunghezza delle uscite: le generazioni sul test misurano in media 3,19 parole contro le 7,07 dei

riferimenti, con un rapporto di 0,45, e soprattutto hanno mediana pari a 0, il che significa che almeno metà delle risposte è vuota, perché il modello emette il token di fine turno immediatamente. Il fenomeno era già visibile durante l’addestramento, dove le generazioni di controllo su 12 item riportavano una lunghezza costantemente intorno a 0,40–0,47 volte quella umana. Le frasi effettivamente prodotte sono di buona qualità — diversi item raggiungono un chrF individuale fra 70 e 100 — ma le risposte vuote entrano nella media con punteggio nullo e dimezzano il risultato aggregato.

Le altre diagnostiche raccolte sul test confermano questa lettura e ne circoscrivono la portata. Il modello riconosce la traduzione corretta fra cinque candidati nel 99% dei casi, contro il 20% del caso, il che mostra che la competenza è presente e viene persa soltanto nel passaggio alla generazione. La dialettalità delle uscite vale 0,702 contro lo 0,88 dei riferimenti umani, quindi il testo prodotto è napoletano ma meno marcato dell’originale. La degenerazione del decoder non è un problema, con ripetizioni lunghe nello 0,4% dei casi contro lo 0,7% degli umani e nessun incipit dominante. Il classificatore avversariale che prova a distinguere le frasi generate da quelle umane raggiunge un’accuratezza di 0,73, ma il controllo che usa la sola lunghezza ne ottiene già 0,672: quasi tutto ciò che rende riconoscibile l’uscita del modello è la sua brevità, non la sua lingua.

3.2 Analisi dei Risultati sul Completamento di dialogo

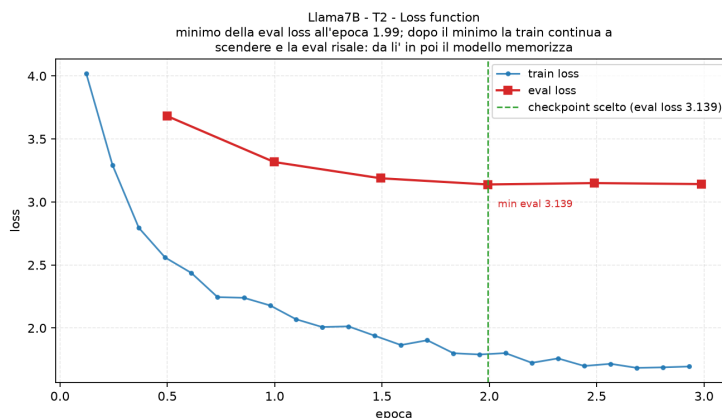


Figura 15: Grafico della Loss-Function per il Fine-Tuning sul Completamento di dialogo con il modello Llama7B

La Figura 15 mostra un andamento regolare. La *train loss* scende da 4,02 a circa 1,70 e la *eval loss* la accompagna in discesa fino al minimo di 3,139, raggiunto all’epoca 1,99, che è anche il punto in cui viene fissato il checkpoint. Dopo il minimo la *eval loss* non risale ma si appiattisce, restando fra 3,14 e 3,15 fino alla fine della terza epoca, mentre la *train loss* continua a scendere lentamente. Il modello, in altre parole, smette semplicemente di migliorare senza cominciare a memorizzare, e l’addestramento arriva a completare le 3 epoche previste senza che l’*early stopping* debba intervenire.

Va notato che il divario fra le due curve resta comunque ampio, circa 1,4 punti di perdita al momento del minimo: il modello si adatta bene agli esempi di addestramento ma generalizza con difficoltà, il che è coerente con il fatto che si tratta di un modello multilingue generalista, la cui competenza sull'italiano è distribuita insieme a quella di molte altre lingue e non concentrata come in un modello dedicato.

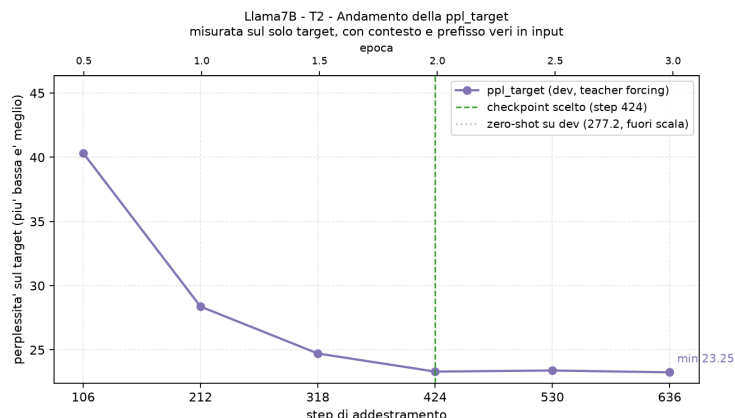


Figura 16: Grafico dell'andamento della perplexity per il Fine-Tuning sul Completamento di dialogo con il modello Llama7B

La curva della perplessità sul *target* in Figura 16, misurata in *teacher forcing* con contesto e prefisso veri in input, conferma la stessa forma della perdita ma su una scala più leggibile e senza alcuna risalita: si scende da 40,33 al passo 106 a 28,37 al passo 212, poi a 24,72 al passo 318 e infine a 23,31 al passo 424, dove viene fissato il checkpoint; le due valutazioni successive restano a 23,39 e 23,25, cioè su un plateau: il modello, anche continuando ad addestrare, non peggiora la propria stima del *target*. Anche in questo caso il valore zero-shot è dichiarato fuori scala dal grafico e vale 277,2, per cui il fine-tuning riduce la perplessità di oltre un ordine di grandezza.

Il fatto che la curva si assesti invece di continuare a scendere indica però che il segnale disponibile è stato sfruttato per intero: con 3400 esempi di addestramento e turni di quattro o cinque parole in mediana, oltre le due epoche non c'è altro da estrarre.

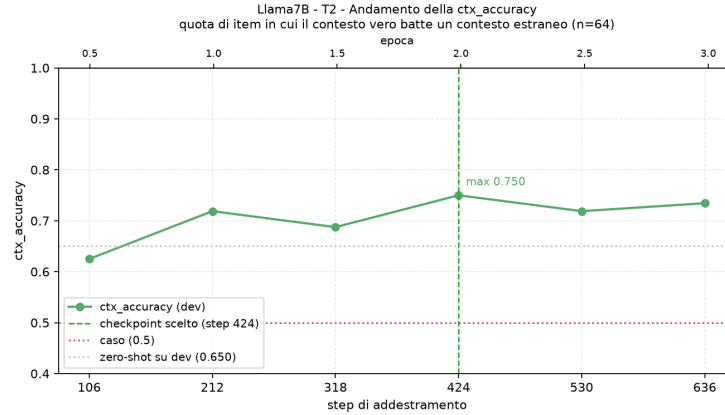


Figura 17: Grafico di `ctx_accuracy` per il Fine-Tuning sul Completamento di dialogo con il modello Llama7B

La Figura 17 riporta l'andamento della `ctx_accuracy` lungo l'addestramento. La metrica misura, su un sottoinsieme di 64 item, la quota di casi in cui il *target* è più probabile con il contesto vero che con un contesto estraneo, dove 0,5 corrisponde al caso e quindi a un contesto del tutto ignorato. La curva parte da 0,625 al passo 106, cioè sotto il valore zero-shot di 0,650, sale a 0,719 al passo 212 e raggiunge il massimo di 0,750 esattamente al passo 424, che è anche il checkpoint selezionato sulla *eval loss*. La coincidenza fra il massimo di questa metrica e il minimo della perdita non è scontata: la selezione operata sulla *eval loss* conserva qui anche il checkpoint che sfrutta meglio il contesto.

Altrettanto significativo è ciò che accade dopo: nelle due valutazioni successive la curva scende solo a 0,719 e 0,734, restando sempre ben sopra il valore zero-shot e molto lontana dal livello del caso. Il vantaggio acquisito, dunque, viene conservato anche proseguendo l'addestramento oltre il checkpoint.

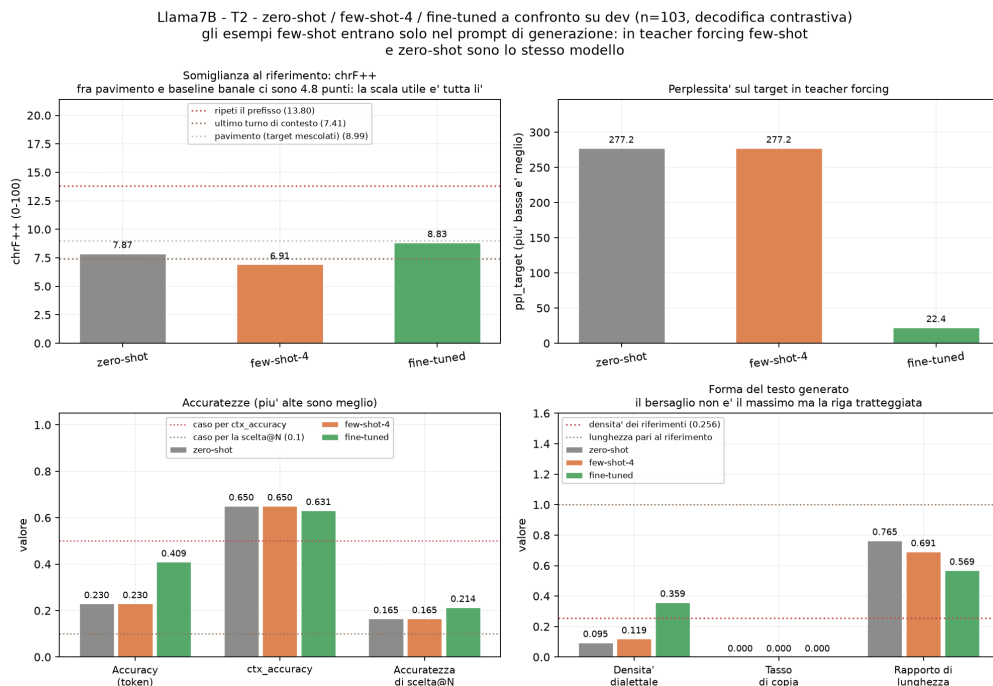


Figura 18: Grafico che mostra la differenza tra approccio zero-shot, few-shot e fine-tuned sul set di sviluppo per il Fine-Tuning sul Completamento di dialogo con il modello Llama7B

Il confronto fra i tre sistemi sul set di sviluppo, in Figura 18, va letto tenendo conto della precisazione riportata nel sottotitolo del grafico: gli esempi *few-shot* entrano soltanto nel prompt di generazione, per cui in *teacher forcing* il sistema *few-shot-4* e quello zero-shot sono letteralmente lo stesso modello e mostrano gli stessi valori. Sul pannello del chrF++ il modello fine-tunato ottiene 8,83 contro il 7,87 dello zero-shot, quindi migliora, mentre il *few-shot* peggiora scendendo a 6,91. Il guadagno resta però confinato sotto il pavimento ottenuto mescolando i *target* (8,99) e molto lontano dalla baseline banale che ripete il prefisso (13,80): come già discusso, su questo task la scala utile della metrica è larga 4,8 punti e nessuno dei tre sistemi la sfrutta davvero. Il dato interessante non è quindi che il fine-tuning migliori il chrF++, ma che il *few-shot* lo peggiori, segno che quattro esempi nel prompt confondono il modello più di quanto lo aiutino.

Gli altri pannelli mostrano guadagni netti e concordi: la perplessità sul *target* passa da 277,2 a 22,4, l'accuratezza a livello di token da 0,230 a 0,409 e l'accuratezza di scelta fra $N = 10$ candidati da 0,165 a 0,214, su un fondo scala di 0,1. Un'eccezione va segnalata per correttezza: la *ctx_accuracy* misurata sull'intero set di sviluppo scende da 0,650 a 0,631, cioè peggiora leggermente, in apparente contrasto con la curva della figura precedente. La discrepanza si spiega con il fatto che la curva di monitoraggio è calcolata su un sottoinsieme di 64 item mentre questo pannello usa tutti i 103 esempi di sviluppo; su un campione così piccolo una differenza di due punti percentuali corrisponde a un paio di item e non è un divario interpretabile. Il pannello sulla forma del testo, infine, mostra la densità dialettale salire da 0,095 a 0,359 contro lo 0,256 dei riferimenti, un tasso di copia del prefisso esattamente nullo in tutti e tre i sistemi, e il rapporto di lunghezza che scende da 0,765 a

0,569 allontanandosi dal bersaglio di 1,0.

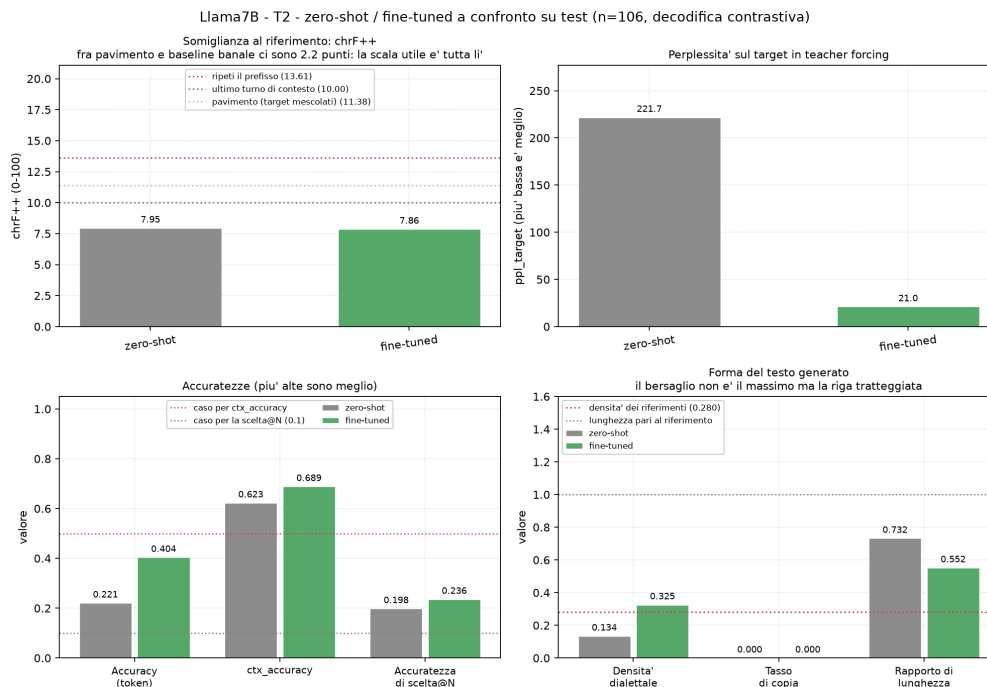


Figura 19: Grafico che mostra la differenza tra approccio zero-shot e fine-tuned sul set di test per il Fine-Tuning sul Completamento di dialogo con il modello Llama7B

La Figura 19 riporta lo stesso confronto sul set di test, dove il sistema *few-shot* non compare perché la sua unica funzione era di controllo in fase di sviluppo. I guadagni in *teacher forcing* sono confermati e in un caso amplificati: la perplessità sul *target* scende da 221,7 a 21,0, l'accuratezza a livello di token sale da 0,221 a 0,404, l'accuratezza di scelta@N da 0,198 a 0,236 e la *ctx_accuracy* da 0,623 a 0,689, cioè qui migliora chiaramente, a differenza di quanto osservato in sviluppo. La densità dialettale passa da 0,134 a 0,325, superando quella dei riferimenti (0,280), e il tasso di copia resta nullo.

Il chrF++, invece, resta sostanzialmente fermo (7,95 contro 7,86) e sotto tutte e tre le righe di riferimento, e il pannello sulla forma del testo spiega perché: il rapporto di lunghezza scende da 0,732 a 0,552. Il modello fine-tunato produce completamenti lunghi poco più della metà del riferimento e, su una metrica di sovrapposizione di caratteri, una risposta troppo corta perde inevitabilmente richiamo indipendentemente da quanto sia appropriata. È l'unico difetto sistematico che le metriche mettono in evidenza in modo concorde.

3.3 Analisi dei Risultati sulla Generazione Libera

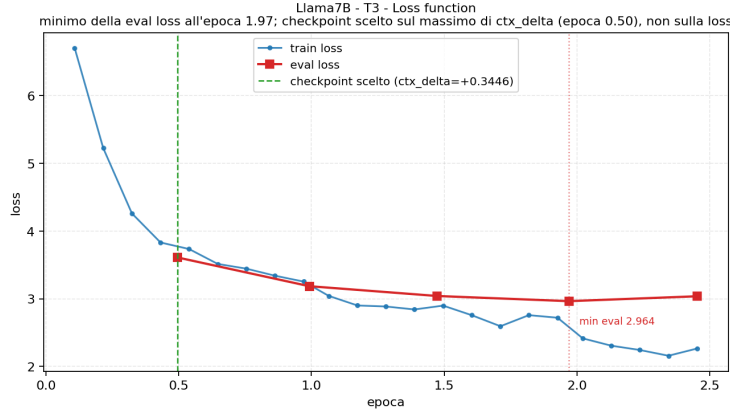


Figura 20: Grafico della Loss-Function per il Fine-Tuning sulla generazione libera con il modello Llama7B

Il grafico in Figura 20 mostra un andamento a prima vista contraddittorio, ed è proprio questa contraddizione a costituire il risultato interessante del task. La *train loss* scende in modo regolare da 6,7 a circa 2,2 e la *eval loss* la accompagna fino al minimo di 2,964, raggiunto all'epoca 1,97, dopo il quale le due curve divergono nel modo classico della memorizzazione. Il checkpoint effettivamente selezionato, però, è quello dell'epoca 0,50, indicato dalla linea verticale tratteggiata: è il punto in cui *ctx_delta* raggiunge il proprio massimo, pari a +0,3446, e si trova quasi un'epoca e mezza prima del minimo della perdita.

La lettura è che le due quantità misurano cose diverse e migliorano in fasi diverse dell'addestramento. La *eval loss* continua a scendere perché il modello impara sempre meglio la forma superficiale dei turni napoletani, ma la pertinenza al contesto, che è la proprietà realmente richiesta dal task, viene raggiunta molto presto e comincia subito dopo a essere erosa: più il modello si specializza sulla stringa esatta del *target*, meno conta per lui quale conversazione l'abbia preceduta. Selezionare il checkpoint sulla perdita avrebbe dunque prodotto un modello dal napoletano più fluente ma più cieco al contesto, ed è esattamente l'errore che la scelta di *ctx_delta* come metrica evita. L'*early stopping* interviene coerentemente alla quinta valutazione, all'epoca 2,5, quattro valutazioni dopo il massimo, interrompendo l'addestramento prima delle 4 epoche previste; il run complessivo è durato circa 58 minuti.

In fase di valutazione, il confronto fra le strategie di decodifica conferma la diagnosi ed è anzi il risultato più netto ottenuto su questo task. La decodifica *greedy* produce un *ctx_delta* di appena 0,026, cioè un modello di fatto indifferente al contesto, e una densità dialettale di 0,034 contro lo 0,084 dei riferimenti umani. La decodifica contestuale, che in inferenza amplifica la differenza fra la distribuzione condizionata al contesto e quella priva di contesto, porta il *ctx_delta* a 1,953 e la densità dialettale a 0,084, cioè esattamente il valore dei riferimenti, con un chrF++ che sale da 11,50 a 12,83. Il parametro γ che regola l'intensità dell'amplificazione mostra un andamento monotono su tutto l'intervallo esplorato: da 0,575 con $\gamma = 0$ fino a 2,579 con $\gamma = 1,2$, con la densità dialettale che cresce

in parallelo da 0,047 a 0,136. Si tratta di un intervento che non riaddestra nulla e agisce direttamente sul difetto osservato, e il fatto che funzioni così bene è un'ulteriore conferma che, almeno su Llama, la cecità al contesto è un problema di decodifica prima ancora che di addestramento.

4 Fine-Tuning attraverso l'utilizzo del modello Gemma

Il terzo modello di base è gemma-3-4b-it [6], che si differenzia dai due precedenti per la dimensione, 4 miliardi di parametri contro i 7 di Minerva e di Llama, il che consente di verificare se il compito richieda davvero la taglia maggiore o se un modello più piccolo ma addestrato più di recente sia sufficiente.

Rispetto all'infrastruttura descritta nel paragrafo precedente due sole cose cambiano, entrambe per vincoli dell'architettura e dell'ambiente di calcolo. La prima è il targeting degli *adapter*: trattandosi di un modello multimodale, i moduli di proiezione dell'attenzione compaiono con lo stesso nome anche nel *vision tower*, e sono stati quindi selezionati per nome completo escludendo esplicitamente tutti i moduli del ramo visivo, che questo task non usa mai. La seconda è la precisione numerica: Gemma-3 nasce in **bf16**, formato che la Tesla T4 non supporta in hardware, e forzarlo in **fp16** produce *overflow* nei logit e una perdita che diverge nelle prime decine di passi; l'addestramento è stato quindi condotto in **fp32**, con un costo in tempo di circa il doppio (5 ore e 37 minuti sul run di T2 contro le 2 ore e 32 minuti dello stesso task su Llama). La pipeline non include gli step di *continued pretraining* e di iniezione lessicale, e l'addestramento riparte direttamente dal modello di base per ciascun task.

4.1 Analisi dei Risultati sulla Traduzione

Il fine-tuning di T1 su Gemma riparte dalla pipeline a tre step descritta nel paragrafo precedente: un primo adapter di *continued pretraining* sul napoletano, un secondo di iniezione lessicale e infine la specializzazione sul task di traduzione, che carica l'adapter dello Step precedente e continua ad aggiornarlo. Lo Step finale ha lavorato su 1134 esempi di addestramento (1016 che provengono dal golden standard e 118 dalla traduzione di Gemma) e 270 di sviluppo, sottocampionati a 200 per le valutazioni periodiche, con 1×10^{-4} , 4 epoche, lunghezza massima di sequenza pari a 512 token. Con un *batch* efficace di 16 esempi si ottengono 71 passi per epoca, 284 passi complessivi e 8 valutazioni intermedie; il controllo sul troncamento dei target e quello sul prefisso tokenizzato non hanno rilevato alcuna anomalia, nè in *train* nè in *dev*. L'addestramento del solo Step finale è durato 1 ora e 46 minuti, contro 2 ore e 20 minuti dell'iniezione lessicale.

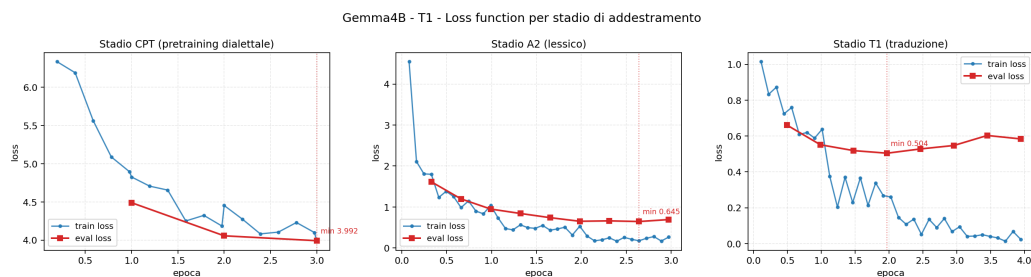


Figura 21: Grafico della Loss-Function per i tre step di addestramento di T1 con il modello Gemma3-4B

La Figura 21 affianca i tre step in altrettanti pannelli, ciascuno con la propria scala verticale; la linea verticale tratteggiata segna in ogni pannello il minimo della eval loss, che sui primi due step coincide con il punto in cui è fissato il checkpoint e sul terzo, come si vedrà, no.

Nel pannello del *continued pretraining* la *train loss* scende da 6.34 a 3.399 lungo tre epoche, con le oscillazioni attese su appena 18 passi di ottimizzazione: il testo napoletano ricostruito dai soli file di addestramento ammonta a 1147 turni, 11229 parole e 21194 token, impacchettati in 41 blocchi da 512. La *eval loss*, misurata una volta per epoca, scende in modo monotono da 4,489 a 4,056 e infine al minimo di 3,992, che coincide con la fine dell' addestramento; in termini di perplessità sul set di sviluppo il passaggio corrisponde a una discesa da 89,05 a 54,17. Nessuna delle due curve mostra segni di risalita: lo Step si è fermato per esaurimento delle epoche previste e non per sovradattamento.

Il pannello dell'iniezione lessica ha la forma tipica di un addestramento su esempi molto corti: la *train loss* crolla da 4,55 a meno di 1 entro la prima epoca e prosegue poi verso 0,27, mentre la *eval loss* scende da 1,614 fino al minimo di 0,645 all'epoca 2,64 e da risalire leggermente a 0,693. Le due curve si separano nettamente nella seconda metà: su 1742 esempi costruiti attorno a poche centinaia di forme dialettali distinte, il modello arriva presto al punto in cui rivedere le stesse coppie non aggiunge informazione. Il checkpoint conservativo è quello del passo 324, dove il chrF di monitoraggio raggiunge 66,45.

Il pannello dello Step di traduzione è quello che richiede più attenzione, perchè il minimo della perdita e il checkpoint effettivamente prossimo non coincidono. La *train loss* parte da 1,01 - un valore già basso, perchè il modello arriva qui con due step di addattamento alle spalle - e scende in scalini successivi fino a 0,02 alla quarta epoca. La *eval loss* scende da 0,661, a 0,551, 0,518 e al minimo di 0,504 all'epoca 1,97, poi risale con regolarità a 0,528, 0,547, 0,603 e 0,584: la distanza fra le due curve alla fine supera i cinque decimi e la forma è quella classica della memorizzazione a partire dalla seconda epoca. Il checkpoint selezionato è però quello dell'epoca 3,45, cioè un'epoca e mezza dopo il minimo della perdita, perchè su T1 il criterio di selezione è il chrF sul set di sviluppo e non la *eval loss*.

Questo scarto non è un'anomalia da correggere ma il risultato più istruttivo del pannello, ed è visibile solo mettendo la Figura 21 accanto alla successiva: mentre la *eval loss* peggiora, il chrF, il BLEU, l'accuratezza a livello di token e le tre metriche lessicali continuano tutte

a migliorare fino all'epoca 3,45. La perdita cresce perchè il modello diventa sempre più sicuro di una singola resa e paga caro ogni deviazione dal target: ma su un task con un unico riferimento corretto ciò che conta è quanto la traduzione prodotta somigli a quel riferimento, e su quello il modello non sta peggiorando affatto. Selezionare il checkpoint sulla perdita, come si fa su T2, avrebbe qui scartato il modello migliore. L'*early stopping* configurato con *patience* pari a 4, non interviene perchè la metrica di selezione non smette mai di salire e l'addestramento completa i 284 passi previsti.

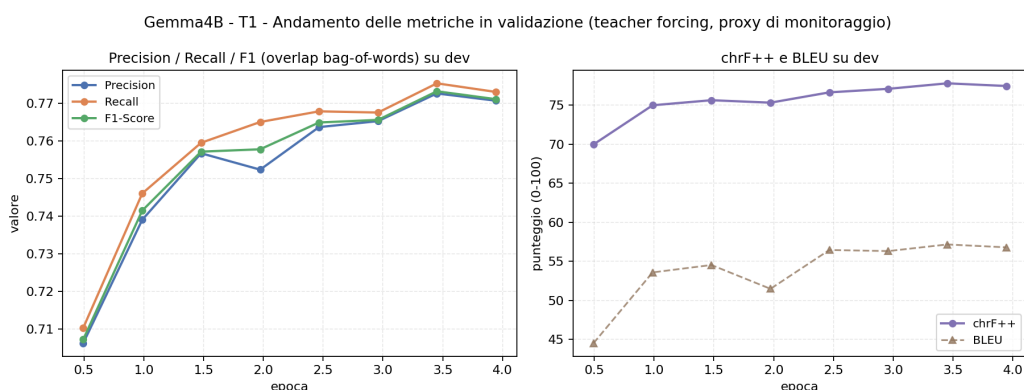


Figura 22: Grafico dell'andamento delle metriche in validazione durante il Fine-Tuning sulla Traduzione con contesto con il modello Gemma3-4B

La Figura 22 riporta le metriche calcolate sul set di sviluppo lungo lo Step di traduzione, in due pannelli e su due scale diverse: a sinistra precision, recall e F1 lessicali a *bag-of-words*, a destra il chrF++ e il BLEU su scala da 0 a 100. Tutte sono misurate in *teacher forcing* e vanno quindi lette come proxy di monitoraggio.

Le tre curve del pannello di sinistra sono di fatto sovrapposte per tutta la durata dell'addestramento, con precision e recall che non divergono mai di più di un punto e mezzo percentuale: il modello non è né sistematicamente prolioso né sistematicamente reticente nelle parole che sceglie. Partono da 0,706, 0,710 e 0,707 alla prima valutazione, salgono a circa 0,757 all'epoca 1,48, restano su quel livello per due valutazioni e raggiungono il massimo di 0,773, 0,775 e 0,773 all'epoca 3,45, cioè esattamente al checkpoint selezionato; l'ultima valutazione scende di due millesimi.

Il pannello di destra mostra la stessa forma, e qui la differenza rispetto a Llama è netta. Il chrF++ sale da 69,96 a 77,78 all'epoca 3,45 e il BLEU da 44,52 a 57,15 alla stessa valutazione: le due metriche raggiungono il massimo insieme, e insieme al massimo delle tre curve lessicali. Su Llama, invece, il chrF++ si era appiattito mentre il BLEU continuava a crescere, segno che il modello stava imparando a riprodurre alla lettera la grafia dei *target* di addestramento. Qui il fatto che le cinque metriche si muovano concordi indica che nelle ultime epoche il modello sta ancora migliorando la traduzione, non solo la sua memoria del riferimento — ed è la lettura che giustifica la scelta di un checkpoint tanto avanzato.

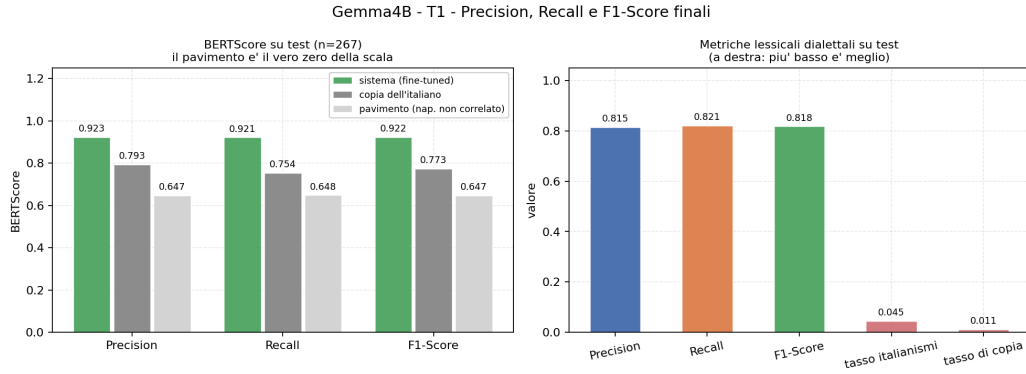


Figura 23: Grafico di confronto delle metriche Precision, Recall e F1-Score finali per il Fine-Tuning sulla Traduzione con contesto con il modello Gemma3-4B

La Figura 23 riporta le metriche finali su test, 267 esempi, in due pannelli. A differenza di quanto accaduto su Llama, qui il file con le metriche finali è stato scaricato dalla sessione di calcolo, per cui entrambi i pannelli riportano precision, recall e F1 per esteso.

Il pannello di sinistra mette il BERTScore del sistema accanto ai suoi due termini di paragone. Il sistema ottiene 0,923 di precision, 0,921 di recall e 0,922 di F1, contro lo 0,773 di F1 della copia dell'italiano lasciato invariato e lo 0,647 del pavimento, cioè il BERTScore calcolato fra riferimenti non correlati fra loro, che è il vero zero della scala e non lo 0,0 dell'asse. Il dato va letto tenendo presente l'avvertenza riportata nel file delle metriche: l'encoder multilingue non conosce il napoletano e tende quindi a premiare la copia dell'italiano, per cui superarla di quindici punti è un margine tutt'altro che scontato.

Il pannello di destra riguarda le metriche lessicali dialettali, quelle che contano quante delle forme dialettali del riferimento il sistema recupera davvero. Precision 0,815, recall 0,821 e F1 0,818: il modello ne recupera più di quattro su cinque, e le due misure sono equilibrate, cioè non ottiene il risultato né inondando la frase di forme dialettali né risparmiandole. Le ultime due barre vanno lette al contrario, come indica il sottotitolo, perché più basse sono e meglio è: il tasso di italianismi vale 0,045 e il tasso di copia 0,011, quest'ultimo particolarmente rilevante perché esclude la spiegazione più banale del punteggio alto, cioè che il sistema stia ricopiando la frase italiana che ha in ingresso.

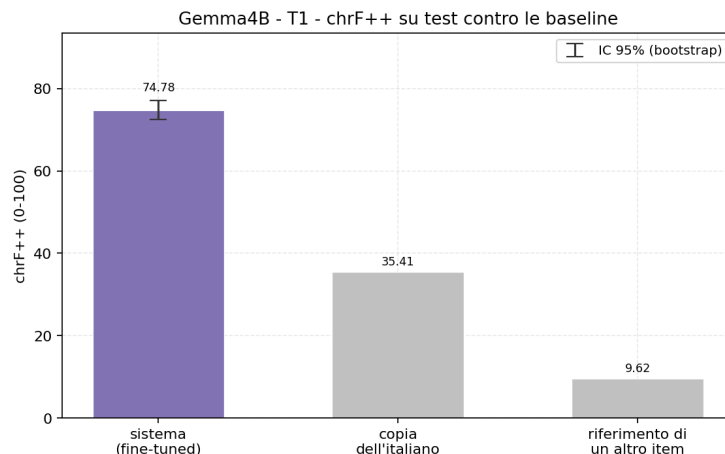


Figura 24: Grafico del chrF++ su test contro le baseline per il Fine-Tuning sulla Traduzione con contesto con il modello Gemma3-4B

La Figura 24 mette il chrF++ del sistema in generazione reale accanto alle due baseline previste per questo task: la copia dell'italiano lasciato invariato, che su questo corpus è tutt'altro che banale perché molti turni coincidono nelle due lingue, e il riferimento di un altro item, che fissa il livello del caso.

Il sistema ottiene 74,78 con un intervallo bootstrap al 95% che va da 72,52 a 77,03, contro il 35,41 della copia dell'italiano e il 9,62 del riferimento estraneo. Il margine è di oltre trentanove punti sulla baseline non banale e l'intervallo di confidenza, ampio poco più di quattro punti, ne resta lontanissimo. È il risultato più netto ottenuto su T1 fra i tre modelli, e va confrontato con il dato che più conta ai fini metodologici: il chrF in *teacher forcing* sul set di sviluppo vale 77,78, quello in generazione autonoma sul test 74,78. I tre punti di scarto fra le due misure dicono che qui la competenza misurata durante l'addestramento si trasferisce quasi per intero alla generazione, mentre su Llama lo stesso confronto registrava un crollo da 76,25 a 36,90.

Le diagnostiche raccolte sul test confermano il quadro in ogni sua parte. Il rapporto di lunghezza vale 1,003 (7,09 parole generate contro le 7,07 dei riferimenti) e il test di Kolmogorov-Smirnov sulle lunghezze dà una statistica di 0,019 con $p = 1,00$, cioè le due distribuzioni sono indistinguibili; nessuna uscita raggiunge il tetto dei 64 token. La dialettalità delle generazioni vale 0,875 contro lo 0,88 dei riferimenti umani e le forme inedite 0,157 contro 0,158, entrambe praticamente sul bersaglio. Il modello riconosce la traduzione corretta fra cinque candidati nel 98% dei casi, contro il 20% del caso. La degenerazione del decoder è assente, con ripetizioni lunghe nello 0,7% dei casi e un incipit dominante al 5,2%, valori che coincidono con quelli umani. Il classificatore avversariale che prova a distinguere le frasi generate da quelle umane raggiunge un'accuratezza di 0,491, cioè il livello del caso, con il controllo umano-contro-umano a 0,505 e quello sulla sola lunghezza a 0,496: su questo test le traduzioni prodotte dal modello non sono statisticamente distinguibili da quelle di riferimento.

4.2 Analisi dei Risultati sul Completamento di dialogo

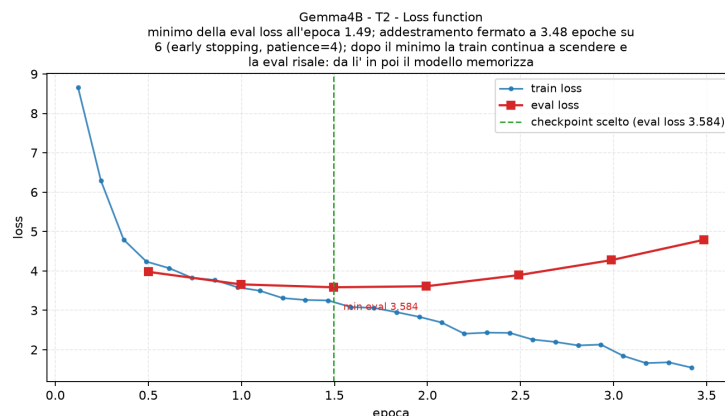


Figura 25: Grafico della Loss-Function per il Fine-Tuning sul Completamento di dialogo con il modello Gemma3-4B

La Figura 25 mostra un caso da manuale di sovradattamento. La *train loss* parte da 8,65 e scende senza mai fermarsi fino a 1,53, mentre la *eval loss* scende solo fino all'epoca 1,49, dove tocca il minimo di 3,584, per poi risalire in modo costante fino a 4,78. Il punto di incrocio delle due curve, poco prima dell'epoca 1, segna il passaggio dall'apprendimento alla memorizzazione: da lì in avanti il modello continua a migliorare sugli esempi che ha già visto e peggiora su quelli che non ha visto. L'*early stopping* interviene all'epoca 3,48 delle 6 previste, quattro valutazioni dopo il minimo, e il checkpoint conservato è quello del passo 318.

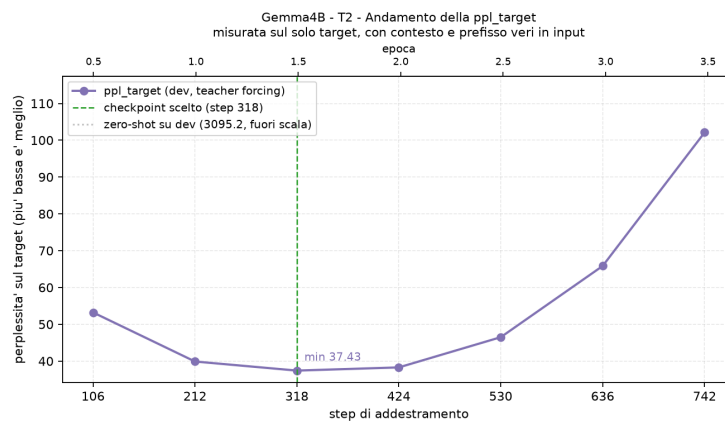


Figura 26: Grafico dell'andamento della perplexity per il Fine-Tuning sul Completamento di dialogo con il modello Gemma3-4B

La curva della perplessità sul *target* in Figura 26, misurata in *teacher forcing* con contesto e prefisso veri in input, riproduce la stessa forma a U vista sulla *eval loss* e ne è di fatto la trasposizione su una scala più leggibile: si scende da 53,22 al passo 106 fino al minimo di 37,43 al passo 318, per poi risalire a 46,50, 65,85 e infine 102,11 all'ultima valutazione. Il dato da tenere presente è però quello che il grafico dichiara fuori scala: la perplessità del

modello zero-shot sugli stessi dati è pari a 3095,2. Il fine-tuning riduce quindi l'incertezza del modello sul *target* di quasi due ordini di grandezza, e persino il punto peggiore della curva resta trenta volte migliore del punto di partenza.

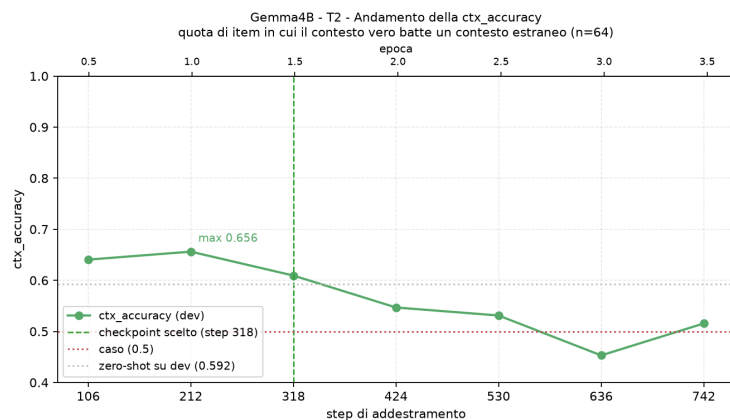


Figura 27: Grafico di `ctx_accuracy` per il Fine-Tuning sul Completamento di dialogo con il modello Gemma3-4B

La Figura 27 racconta invece la parte meno favorevole del run, ed è il motivo per cui questa metrica viene riportata accanto alle altre invece di essere omessa. La `ctx_accuracy` misura, su un sottoinsieme di 64 item, la quota di casi in cui il *target* è più probabile con il contesto vero che con un contesto estraneo: un valore di 0,5 significa contesto del tutto ignorato. Il modello zero-shot si colloca a 0,592, il massimo lungo l'addestramento è 0,656 al passo 212, e il checkpoint effettivamente selezionato, al passo 318, vale 0,609. Il guadagno rispetto al punto di partenza è quindi di appena 1,7 punti percentuali, e nella seconda metà dell'addestramento la curva scende fino a 0,453, cioè sotto il livello del caso. Il fenomeno è lo stesso osservato su T3: mentre il modello si specializza sulla forma dei turni, il contesto conversazionale conta progressivamente di meno nel determinare la sua previsione. Va inoltre notato che il checkpoint scelto sulla *eval loss* non coincide con il massimo di `ctx_accuracy`, il che suggerisce che anche su T2 una selezione basata su questa metrica, come quella adottata su T3, sarebbe stata più coerente con l'obiettivo del task.

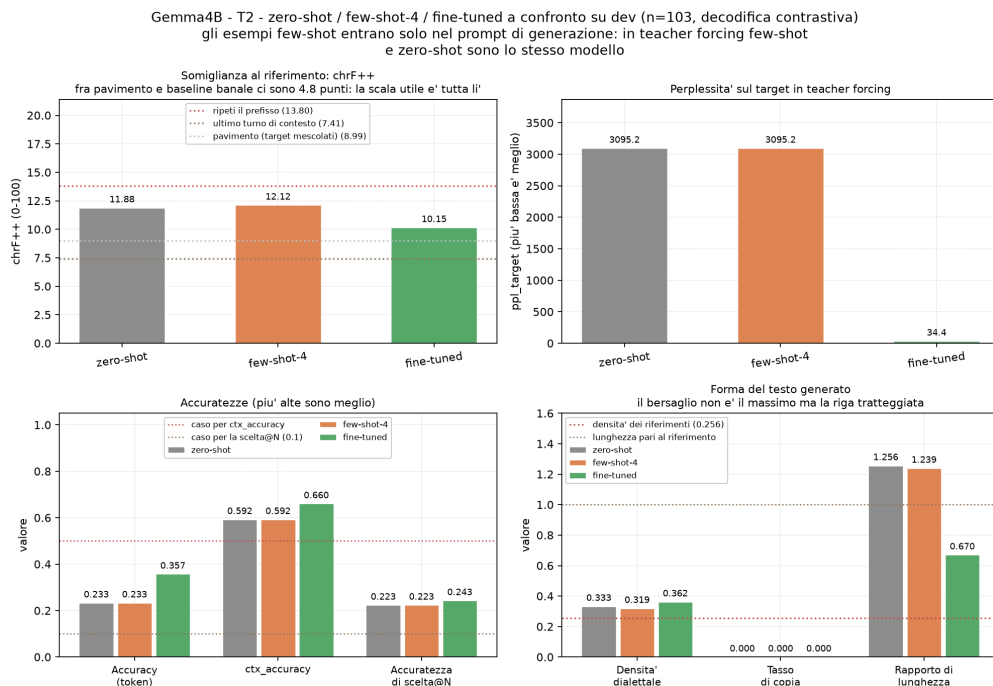


Figura 28: Grafico che mostra la differenza tra approccio zero-shot, few-shot e fine-tuned sul set di sviluppo per il Fine-Tuning sul Completamento di dialogo con il modello Gemma3-4B

Il confronto fra i tre sistemi sul set di sviluppo, in Figura 28, va letto tenendo conto di una precisazione riportata nel sottotitolo del grafico stesso: gli esempi *few-shot* entrano soltanto nel prompt di generazione, per cui in *teacher forcing* il sistema *few-shot-4* e quello zero-shot sono letteralmente lo stesso modello e mostrano gli stessi valori. Il pannello del chrF++ è quello che a prima vista contraddice tutto il resto: il modello fine-tunato ottiene 10,15 contro l'11,88 dello zero-shot e il 12,12 del *few-shot*, quindi peggiora. Il pannello va però letto insieme alle tre righe tratteggiate che riporta: il pavimento ottenuto mescolando i *target* vale 8,99 e la baseline banale che ripete il prefisso vale 13,80, per cui l'intera scala utile della metrica su questo task è larga 4,8 punti e nessuno dei tre sistemi raggiunge la baseline banale. Su un task in cui il riferimento è uno fra molti completamenti validi, il chrF++ non è in grado di ordinare i sistemi.

Gli altri tre pannelli, che misurano quantità meno ambigue, concordano tutti nella direzione opposta. La perplessità sul *target* passa da 3095,2 a 34,4, l'accuratezza a livello di token da 0,233 a 0,357, la *ctx_accuracy* da 0,592 a 0,660 e l'accuratezza di scelta fra $N = 10$ candidati da 0,223 a 0,243, su un fondo scala di 0,1. Il pannello sulla forma del testo generato spiega infine da dove venga il calo del chrF++: il rapporto di lunghezza passa da 1,256 a 0,670, quando il bersaglio è 1,0. Il modello zero-shot produce turni troppo lunghi, quello fine-tunato ha imparato che i turni di questo corpus sono brevi ma ha corretto oltre il bersaglio, e una risposta più corta del riferimento perde inevitabilmente richiamo di caratteri. La densità dialettale, intanto, sale da 0,333 a 0,362 contro lo 0,256 dei riferimenti, e il tasso di copia del prefisso resta esattamente nullo in tutti e tre i sistemi, il che esclude la spiegazione più banale, cioè che il modello si limiti a ricopiare ciò che ha

in input.

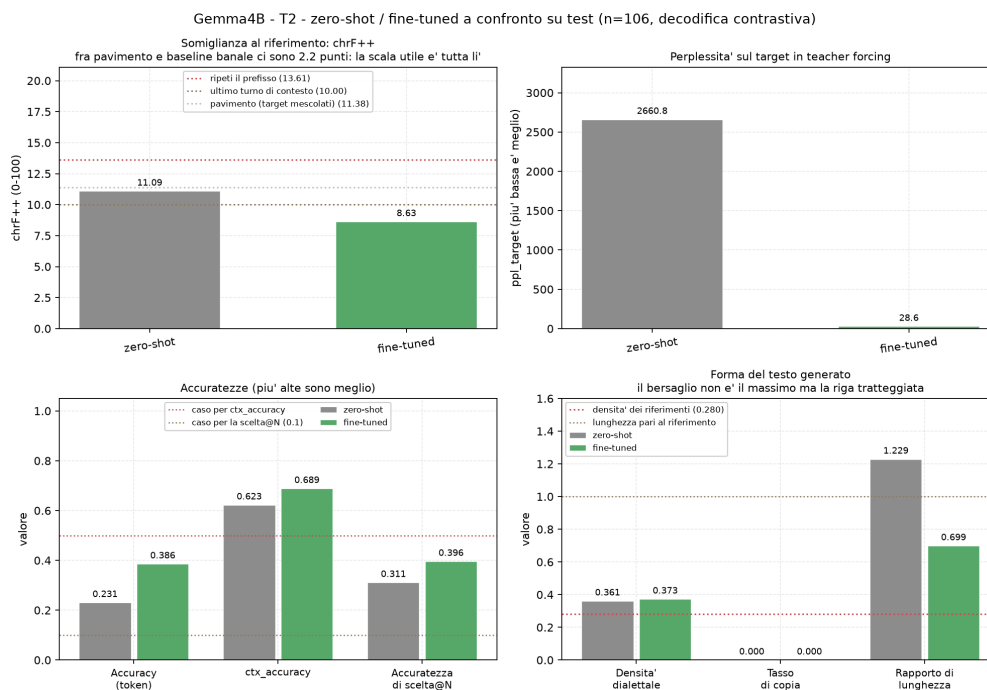


Figura 29: Grafico che mostra la differenza tra approccio zero-shot e fine-tuned sul set di test per il Fine-Tuning sul Completamento di dialogo con il modello Gemma3-4B

La Figura 29 riporta lo stesso confronto sul set di test, dove il sistema *few-shot* non compare perché la sua unica funzione era di controllo in fase di sviluppo. Il quadro è confermato in ogni sua parte: la perplessità sul *target* scende da 2660,8 a 28,6, l'accuratezza a livello di token sale da 0,231 a 0,386, la *ctx_accuracy* da 0,623 a 0,689 e l'accuratezza di scelta@N da 0,311 a 0,396, con un miglioramento più marcato di quello osservato in sviluppo. Il chrF++ scende ancora, da 11,09 a 8,63, per la stessa ragione vista sopra, con il rapporto di lunghezza che passa da 1,229 a 0,699, mentre la densità dialettale rimane superiore a quella dei riferimenti (0,373 contro 0,280) e il tasso di copia nullo. La conclusione che se ne trae è duplice: il fine-tuning ha prodotto un adattamento reale e verificabile su tutte le misure che non dipendono dalla scelta di un riferimento singolo, e la lunghezza delle risposte è il parametro su cui il modello va ancora calibrato, essendo l'unico difetto sistematico che le metriche mettono in evidenza in modo concorde.

4.3 Analisi dei Risultati sulla Generazione Libera

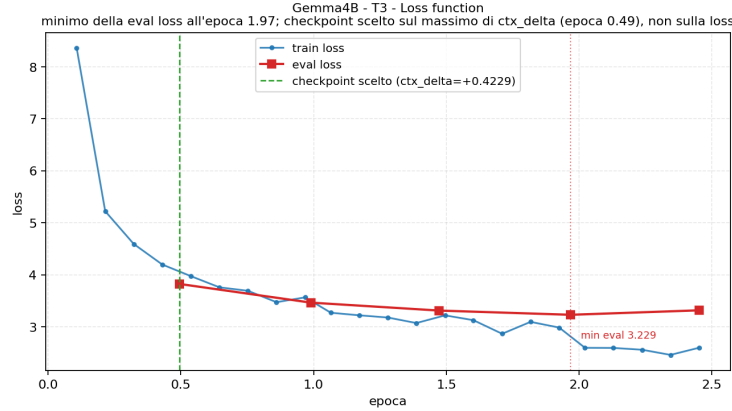


Figura 30: Grafico della Loss-Function per il Fine-Tuning sulla generazione libera con il modello Gemma3-4B

Nella Figura 30 il minimo della perdita e il punto in cui viene fissato il checkpoint non coincidono. La *train loss* scende da 8,35 a circa 2,45 e la *eval loss* raggiunge il minimo di 3,229 all'epoca 1,97, mentre il checkpoint selezionato è quello dell'epoca 0,49, dove *ctx_delta* tocca il massimo di +0,4229. L'ottimo di pertinenza contestuale si colloca dunque a metà della prima epoca, circa un'epoca e mezza prima del minimo della perdita, e l'*early stopping* interrompe il run all'epoca 2,5 delle 4 previste, dopo circa 117 minuti.

Anche in fase di valutazione il quadro è coerente. La decodifica *greedy* produce turni lunghi più del doppio del riferimento (rapporto 2,337) e un *ctx_delta* di appena 0,278, mentre la decodifica contestuale lo porta a 1,796 riducendo al contempo il rapporto di lunghezza a 1,802. L'esplorazione del parametro γ mostra un andamento monotono, da 0,732 con $\gamma = 0$ fino a 2,780 con $\gamma = 1,2$, con la densità dialettale che passa da 0,047 a 0,096 contro lo 0,097 dei riferimenti umani e il rapporto di lunghezza che converge a 1,101, cioè quasi esattamente il bersaglio. Il valore $\gamma = 1,2$ risulta quindi simultaneamente il migliore su pertinenza, dialettalità e lunghezza, il che rende la decodifica contestuale l'intervento a costo nullo più efficace fra quelli sperimentati su questo modello.

5 Valutazione umana su scala Likert

Tutte le misure discusse finora sono automatiche e, come si è visto ripetutamente, nessuna di esse è in grado da sola di dire se una frase napoletana sia *buona*: il chrF++ premia la sovrapposizione di caratteri con un riferimento che è solo uno fra i molti completamente validi, la perplessità misura quanto il modello sia sicuro del bersaglio e non quanto quel bersaglio sia ben scritto, la densità dialettale conta le forme dialettali senza sapere se siano usate bene. Per colmare questa lacuna è stata condotta una valutazione umana sistematica sulle tre dimensioni previste dal progetto: **naturalzza dialettale**, **accuratezza grammaticale** e **coerenza lessicale**, ciascuna su scala Likert a cinque punti.

Sono stati valutati sei sistemi, ovvero ciascuno dei tre modelli su ciascuno dei due task generativi. Il task di traduzione (T1) resta fuori da questa valutazione perché dispone già di un riferimento unico per frase e quindi di metriche automatiche interpretabili, mentre T2 e T3 sono esattamente i due casi in cui le metriche automatiche si sono rivelate poco informative.

5.1 Scelta dei sistemi da valutare

Per ciascun modello e per ciascun task è stata scelta la configurazione che risponde meglio al contesto, non quella con il chrF più alto. La ragione è quella già emersa dalle analisi dei singoli modelli: su questi due task il chrF misura la sovrapposizione con un solo riferimento fra i molti completamenti ugualmente validi, e finisce quindi per premiare chi ricalca la forma superficiale del bersaglio, non chi ha capito la conversazione. Il criterio adottato è invece la coppia di metriche che misura direttamente quanto il modello cambi risposta al cambiare del contesto, cioè la `ctx_accuracy` su T2 e il `ctx_delta` su T3.

Su T2 la scelta è di fatto obbligata, perché lo split di test espone un solo sistema fine-tunato per modello. Su T3, dove convivono quattro strategie di decodifica, la scelta cade in tutti e tre i casi sulla decodifica contestuale, che è anche quella con il divario più netto rispetto alle alternative: su Minerva il `ctx_delta` passa da 1,048 in *greedy* a 2,385, su Gemma da 0,278 a 1,796 e su Llama, il caso più estremo, da 0,026 a 1,953, cioè da un modello praticamente cieco al contesto a uno che lo sfrutta. Le varianti γ da 0,0 a 1,2 non entrano nel confronto perché sono un’ablazione condotta su 64 item invece che sui 168 del test, e non sarebbero confrontabili con le altre.

Task	Modello	Configurazione	Criterio
T2	Minerva7B	fine-tuned, <i>beam</i>	<code>ctx_accuracy</code> 0,660
T2	Llama7B	fine-tuned, contrastiva	<code>ctx_accuracy</code> 0,689
T2	Gemma3-4B	fine-tuned, contrastiva	<code>ctx_accuracy</code> 0,689; scelta@N 0,396
T3	Minerva7B	fine-tuned, contestuale	<code>ctx_delta</code> 2,385 (1,048 in <i>greedy</i>)
T3	Llama7B	fine-tuned, contestuale	<code>ctx_delta</code> 1,953 (0,026 in <i>greedy</i>)
T3	Gemma3-4B	fine-tuned, contestuale	<code>ctx_delta</code> 1,796 (0,278 in <i>greedy</i>)

Tabella 1: I sei sistemi sottoposti a valutazione umana, uno per modello e per task, con il criterio che ne ha determinato la scelta.

5.2 Protocollo di annotazione

Il disegno sperimentale ha due caratteristiche. La prima è che è **appaio** fra modelli: da ciascun task sono state estratte 20 conversazioni a caso, e per ognuna di esse è stata inserita nella scheda la risposta di tutti e tre i modelli. Poiché i tre modelli sono stati valutati sullo stesso split di test, con gli stessi identificativi e nello stesso ordine, questo è possibile senza alcuna forzatura, e ha una conseguenza importante: una differenza di punteggio fra due modelli è attribuibile al modello e non alle frasi che gli sono capitate in sorte, dato che le frasi di partenza sono le stesse. Le differenze fra modelli

vengono perciò calcolate conversazione per conversazione, il che elimina la variabilità dovuta alla difficoltà del singolo contesto e restringe sensibilmente gli intervalli di confidenza.

La seconda caratteristica è che le condizioni sono **mescolate**. Alle 60 frasi generate per task sono stati affiancati 15 turni umani presi dal riferimento del corpus, per un totale di 150 item, disposti in ordine casuale e reciprocamente diverso per i due annotatori, così da neutralizzare gli effetti di stanchezza e di trascinamento fra item consecutivi. I turni umani non sono un settimo sistema in gara ma un **termine di paragone**: fissano il tetto realistico della scala, dato che nemmeno un parlante nativo trascritto da parlato spontaneo ottiene punteggi pieni. Le conversazioni da cui provengono sono disgiunte da quelle usate per i modelli, così che nessun contesto compaia sia con la frase generata sia con quella umana.

Ogni item della scheda dichiara da quale sistema proviene la frase, così che i punteggi restino rileggibili accanto al modello che li ha meritati e il campione sia verificabile a posteriori; in fase di analisi l'etichetta viene riconfrontata con la chiave, e una discordanza interrompe l'elaborazione anziché attribuire i voti al sistema sbagliato. La scelta ha però un costo, di cui va tenuto conto nel leggere i risultati: sapendo che una frase proviene da un modello, l'annotatore può essere sistematicamente più severo di quanto sarebbe stato con lo stesso testo attribuito a un parlante reale, e il confronto fra i sistemi ne risente. Le linee guida chiedono per questo di giudicare il testo e non l'etichetta, e la procedura prevede in alternativa una modalità in cieco, in cui la provenienza è nascosta all'annotatore e resta soltanto nella chiave.

Ciascun annotatore ha lavorato in autonomia, senza concordare i punteggi con l'altro: l'accordo fra le due schede è esso stesso un risultato da misurare, e concordare le valutazioni durante il lavoro lo renderebbe privo di significato. Per ogni item l'annotatore vede i turni di conversazione precedenti, l'istruzione che descrive che cosa doveva fare chi parla e infine la frase da valutare; in T2, dove la frase è un completamento, il prefisso già dato viene mostrato distinto dalla parte generata, ma il giudizio grammaticale è espresso sul turno intero.

Le tre dimensioni sono state definite come segue:

- **Naturalzza dialettale**: quanto la frase suoni come napoletano realmente parlato, non come italiano travestito né come napoletano di maniera. Il punteggio 1 corrisponde a un testo che napoletano non è, il 3 a un napoletano riconoscibile ma legnoso e più scritto che parlato, il 5 a un enunciato indistinguibile da quello di un parlante nativo in conversazione spontanea.
- **Accuratezza grammaticale**: correttezza morfologica e sintattica, ovvero articoli e loro forme contratte, raddoppiamento fonosintattico, accordi, coniugazioni, clitici e preposizioni articolate. Il punteggio 1 corrisponde a una frase agrammaticale, il 3 a un errore evidente o a più errori minori, il 5 all'assenza di errori. Le varianti ortografiche non sono penalizzate: il napoletano non ha una grafia standard, e forme come *chiù* e *cchiù* sono entrambe legittime.
- **Coerenza lessicale**: se le parole scelte stiano insieme fra loro e con il contesto, cioè registro uniforme, assenza di italianismi stonati in mezzo al dialetto e frase che nel suo insieme sta in piedi come risposta a *quella* conversazione e non a una qualunque.

Il punteggio 1 è riservato alle risposte fuori tema o buone per qualsiasi contesto, il 3 a scelte accettabili ma discutibili, il 5 a un lessico del tutto appropriato. Le risposte che ripetono più volte la stessa cosa vengono penalizzate su questa dimensione.

Le tre dimensioni sono deliberatamente indipendenti fra loro: una frase può essere grammaticalmente ineccepibile ma lessicalmente fuori contesto, oppure naturalissima e insieme scorretta, e tenerle separate permette di individuare quale dei tre aspetti il fine-tuning abbia effettivamente trasferito a ciascun modello.

5.3 Analisi dei risultati

Le due schede compilate in autonomia coprono 150 item ciascuna, 75 per task, corrispondenti alle 20 conversazioni per modello più i 15 turni umani di riferimento. Ogni valore riportato nei grafici che seguono è quindi la media dei due giudizi indipendenti espressi sulla stessa conversazione, e ogni gruppo di 20 conversazioni raccoglie 40 giudizi effettivi; per il riferimento umano, che poggia su 15 conversazioni, i giudizi sono 30.

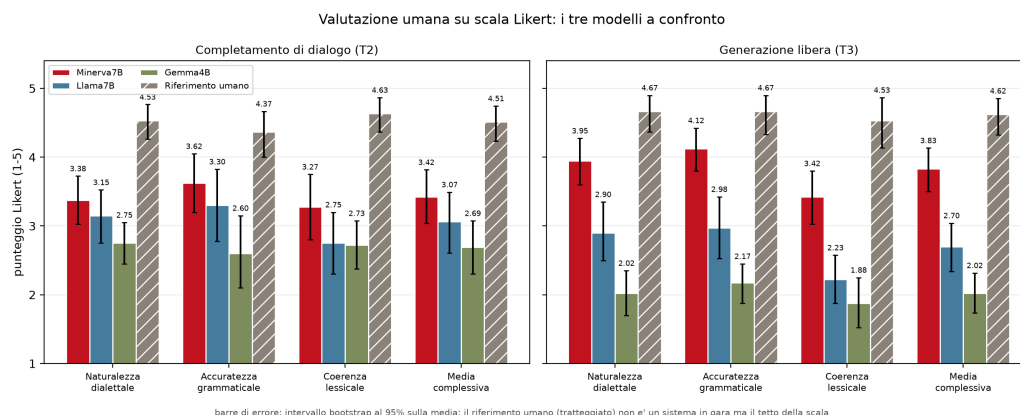


Figura 31: Punteggi Likert medi sulle tre dimensioni, per ciascuno dei tre modelli e per i turni umani di riferimento. Le barre di errore riportano l'intervallo bootstrap al 95% sulla media.

La Figura 31 dedica un pannello a ciascun task e mantiene l'asse verticale ancorato all'intera scala, da 1 a 5, così che le altezze siano leggibili in valore assoluto e non soltanto l'una rispetto all'altra. Dentro ogni pannello i quattro gruppi di barre corrispondono alle tre dimensioni e alla loro media complessiva; dentro ogni gruppo le prime tre barre sono i modelli e la quarta, tratteggiata, è il riferimento umano. Le barre di errore riportano l'intervallo bootstrap al 95% sulla media del gruppo.

Sul completamento di dialogo i tre modelli si distribuiscono in una fascia stretta, fra 2,60 e 3,62, e le medie complessive valgono 3,42 per Minerva, 3,07 per Llama e 2,69 per Gemma, con intervalli che si sovrappongono in larga parte. Sulla generazione libera lo stesso ordinamento si ripresenta, ma le distanze si allargano: 3,83, 2,70 e 2,02, con intervalli che non si toccano. Il confronto fra i due pannelli mostra inoltre che il passaggio da T2 a T3 non agisce nella stessa direzione sui tre sistemi: Minerva guadagna quattro decimi di punto sulla media complessiva, mentre Llama ne perde 0,37 e Gemma 0,67. Il compito

meno vincolato, quello che lascia al modello la scelta di che cosa dire e non soltanto di come completare un turno già avviato, premia dunque un sistema e penalizza gli altri due.

Una regolarità attraversa quasi tutte le colonne: la coerenza lessicale è la dimensione più bassa per tutti e tre i modelli su T3 e per Minerva e Llama su T2. Fanno eccezione Gemma su T2, dove i tre valori sono di fatto indistinguibili (2,75, 2,60 e 2,73, con intervalli che si contengono a vicenda), e il riferimento umano su T2, dove il lessico è invece la dimensione più alta (4,63). Il distacco fra grammatica e lessico è particolarmente marcato per Minerva su T3, dove i due valori valgono 4,12 e 3,42: il modello scrive turni formalmente corretti più spesso di quanto scelga le parole giuste per quella conversazione.

Il riferimento umano si colloca su entrambi i task fra 4,37 e 4,67 e non tocca mai il 5. Nemmeno un turno realmente pronunciato da un parlante nativo e trascritto da parlato spontaneo ottiene punteggi pieni, ed è la ragione per cui la barra tratteggiata va letta come tetto realistico della scala e non come un settimo sistema in gara. Rispetto a questo tetto il divario del migliore fra i tre modelli vale 1,09 punti sulla media complessiva di T2 e 0,79 su quella di T3; va ricordato, nel leggerlo, che l'annotazione non è stata condotta in cieco e che una parte di questa distanza può derivare dalla maggiore severità con cui si giudica una frase dichiaratamente prodotta da un modello.

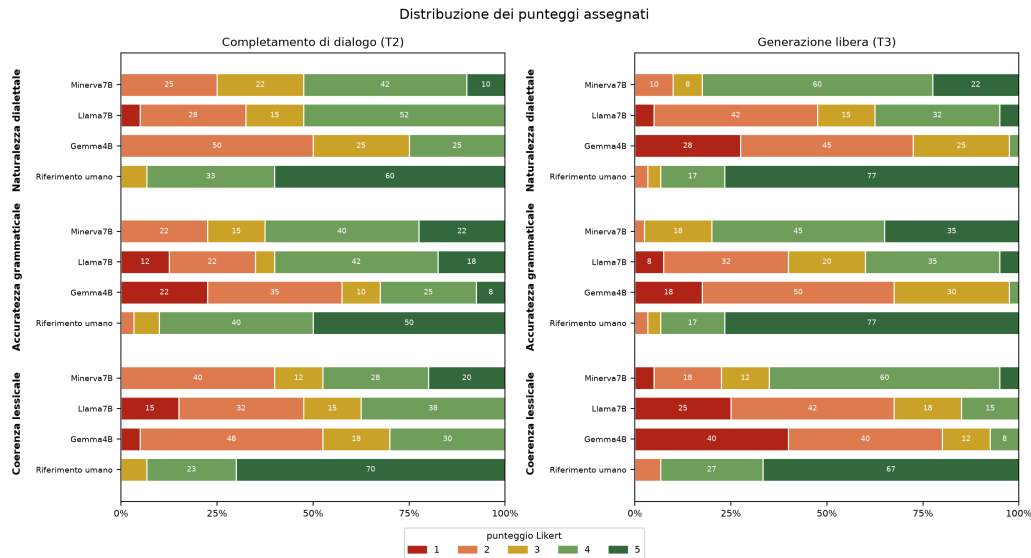


Figura 32: Distribuzione dei punteggi assegnati dai due annotatori, per modello e per dimensione.

La Figura 32 scompone le stesse valutazioni nella loro forma. Ogni riga è una barra impilata al 100% in cui i cinque segmenti, dal rosso scuro per il punteggio 1 al verde scuro per il 5, riportano la quota di giudizi caduta su ciascun valore della scala; le righe sono raggruppate per dimensione e, dentro ciascuna dimensione, per sistema, con i due task affiancati come nella figura precedente. È l'informazione che la media da sola non conserva, dato che distribuzioni molto diverse possono produrre lo stesso valore medio.

Il caso più evidente è l'accuratezza grammaticale su T2. La media di Llama, pari a 3,30, letta da sola suggerirebbe una prestazione mediocre e uniforme; la barra mostra invece due addensamenti opposti, il 12% di giudizi 1 e il 22% di giudizi 2 da un lato, il 42% di 4 e il 18% di 5 dall'altro, con appena il 5% di valutazioni intermedie. Il modello alterna cioè turni grammaticalmente ineccepibili a turni gravemente scorretti, invece di produrre un dialetto costantemente approssimativo. Sulla stessa dimensione Gemma ha una forma analoga, con il 22% di 1 e il 35% di 2 accanto a un 25% di 4 e un 8% di 5. È da qui che vengono le deviazioni standard superiori a 1,25 punti che entrambi i sistemi mostrano su questa casella, le più alte dell'intera valutazione.

All'estremo opposto stanno le righe di Gemma su T3, dove non compare alcun giudizio 5 su nessuna delle tre dimensioni: sulla naturalezza il valore 4 raccoglie un solo giudizio su quaranta e il 73% delle valutazioni si ferma a 1 o 2, mentre sulla coerenza lessicale il 40% dei giudizi è 1 e un altro 40% è 2. Qui la media bassa non è l'effetto di pochi casi disastrosi che trascinano il valore verso il basso, ma descrive la distribuzione nella sua interezza. Minerva su T3, per contrasto, concentra il 60% dei giudizi di naturalezza sul valore 4 e il 22% sul 5, cioè oltre otto giudizi su dieci nella metà alta della scala; sulla coerenza lessicale mantiene la stessa concentrazione sul 4 ma vi affianca un 23% di giudizi 1 o 2, e la differenza fra le due dimensioni si gioca quindi per intero nella coda bassa.

Le righe del riferimento umano hanno una forma che nessun sistema replica. Su T3 il 77% dei giudizi è 5 sia sulla naturalezza sia sulla grammatica e la coda inferiore si ferma a un singolo item valutato 2; su T2 la concentrazione è più moderata, con il 60% di 5 sulla naturalezza e il 70% sulla coerenza lessicale. Che anche queste barre conservino una coda visibile conferma quanto già osservato sulle medie, cioè che la scala non satura nemmeno sul parlato autentico.

I confronti fra modelli sono accompagnati da un intervallo di confidenza bootstrap al 95% calcolato per percentili anziché assumendo la normalità, che su una scala ordinale a cinque punti non sarebbe giustificata, e sono *appaiati*, cioè costruiti sulle differenze osservate conversazione per conversazione. Il confronto con il riferimento umano è invece necessariamente non appaiato, perché per costruzione i turni umani provengono da conversazioni diverse da quelle sottoposte ai modelli, e i suoi intervalli risultano quindi più larghi a parità di numero di item.

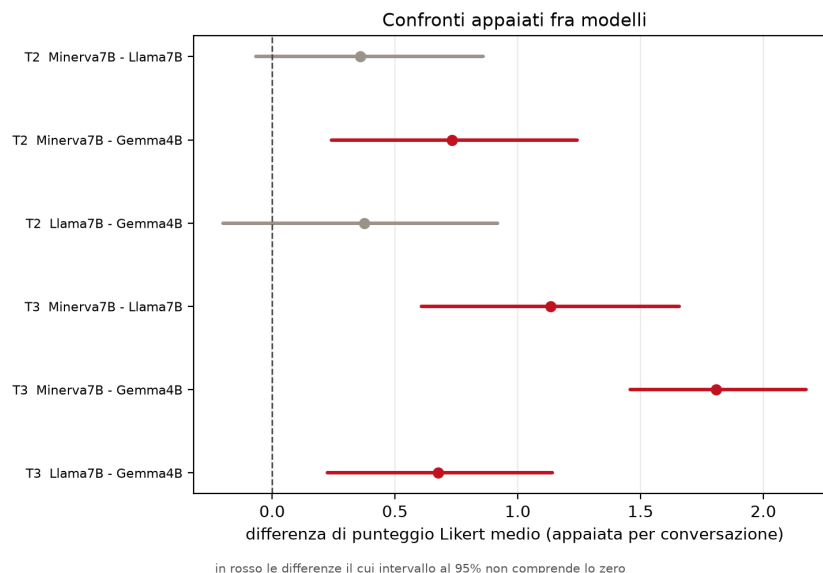


Figura 33: Differenze appaiate fra modelli sulla media complessiva delle tre dimensioni: quando l'intervallo attraversa lo zero i due modelli non sono distinguibili.

La Figura 33 riporta le sei differenze appaiate fra coppie di modelli, tre per ciascun task. Ogni riga mostra il punto stimato della differenza fra le medie complessive e il segmento del suo intervallo bootstrap al 95%; la linea verticale tratteggiata segna lo zero, le righe in rosso sono quelle il cui intervallo non lo comprende e quelle in grigio indicano coppie che i dati raccolti non consentono di separare. Poiché i tre modelli hanno risposto alle stesse 20 conversazioni, la differenza è calcolata conversazione per conversazione, il che toglie dalla stima la variabilità dovuta alla difficoltà del singolo contesto.

Le tre righe di T2 occupano posizioni simili ma una sola è rossa: Minerva meno Llama vale +0,36 con intervallo $[-0,07; +0,86]$ e Llama meno Gemma vale +0,37 con intervallo $[-0,20; +0,92]$, e in entrambi i casi lo zero cade dentro l'intervallo. Solo il confronto fra gli estremi della classifica, Minerva meno Gemma, produce una differenza separabile dallo zero: +0,73 con intervallo $[+0,24; +1,24]$. Su questo task, quindi, i dati raccolti autorizzano a dire che il primo e il terzo sistema sono diversi, ma non a ordinare le coppie contigue.

Le tre righe di T3 sono invece tutte rosse e tutte spostate a destra: +1,13 con intervallo $[+0,61; +1,66]$ per Minerva su Llama, +0,68 con intervallo $[+0,23; +1,14]$ per Llama su Gemma e +1,81 con intervallo $[+1,46; +2,17]$ per Minerva su Gemma, quest'ultimo il segmento più corto dei sei. L'ordinamento dei tre modelli è dunque completamente risolto sulla generazione libera e solo in parte sul completamento di dialogo. Il confronto fra i due blocchi dà anche la misura della risoluzione dello strumento: con 20 conversazioni e due annotatori la soglia oltre la quale una differenza si stacca dallo zero si colloca poco sotto i sette decimi di punto, dato che +0,68 risulta separabile e +0,37 no.

Dimensione	Minerva7B	Llama7B	Gemma3-4B	Riferimento umano
<i>Completamento di dialogo (T2)</i>				
Naturalzza dialettale	3,38 (0,83)	3,15 (0,93)	2,75 (0,72)	4,53 (0,52)
Accuratezza grammaticale	3,62 (0,99)	3,30 (1,26)	2,60 (1,25)	4,37 (0,69)
Coerenza lessicale	3,27 (1,11)	2,75 (1,06)	2,73 (0,83)	4,63 (0,52)
<i>Media complessiva</i>	3,42 (0,91)	3,07 (1,02)	2,69 (0,90)	4,51 (0,52)
<i>Generazione libera (T3)</i>				
Naturalzza dialettale	3,95 (0,81)	2,90 (1,01)	2,02 (0,75)	4,67 (0,56)
Accuratezza grammaticale	4,12 (0,74)	2,98 (1,03)	2,17 (0,69)	4,67 (0,59)
Coerenza lessicale	3,42 (0,91)	2,23 (0,82)	1,88 (0,87)	4,53 (0,74)
<i>Media complessiva</i>	3,83 (0,74)	2,70 (0,81)	2,02 (0,69)	4,62 (0,57)

Tabella 2: Valutazione umana su scala Likert 1-5: media e, fra parentesi, deviazione standard, per ciascun modello e per i turni umani di riferimento. In grassetto il valore più alto fra i tre modelli su ogni riga.

Confronto	n	Δ media	IC 95%
<i>Completamento di dialogo (T2)</i>			
Minerva7B – Llama7B	20	+0,36	[−0,07; +0,86]
Minerva7B – Gemma3-4B	20	+0,73*	[+0,24; +1,24]
Llama7B – Gemma3-4B	20	+0,37	[−0,20; +0,92]
<i>Generazione libera (T3)</i>			
Minerva7B – Llama7B	20	+1,13*	[+0,61; +1,66]
Minerva7B – Gemma3-4B	20	+1,81*	[+1,46; +2,17]
Llama7B – Gemma3-4B	20	+0,68*	[+0,23; +1,14]

Tabella 3: Confronti appaiati fra modelli sulla media complessiva delle tre dimensioni: i modelli hanno risposto alle stesse conversazioni, quindi la differenza è calcolata conversazione per conversazione. L’asterisco indica un intervallo bootstrap al 95% che non comprende lo zero.

Le Tabelle 2 e 3 riportano gli stessi dati in forma numerica, con le deviazioni standard che i grafici non mostrano e con gli estremi esatti degli intervalli.

L’affidabilità dell’annotazione è misurata con tre indici complementari: l’ α di Krippendorff su scala ordinale, che tiene conto della distanza fra le categorie e non solo della loro identità; la κ di Cohen con pesi quadratici, che penalizza un disaccordo fra 1 e 5 molto più di uno fra 4 e 5; e il ρ di Spearman, che misura se i due annotatori ordinano allo stesso modo gli item anche quando usano la scala con severità diversa. Accanto a questi viene riportata la quota di giudizi che si discostano di al più un punto, che è la lettura più immediata dell’accordo su una scala corta. I valori ottenuti sono raccolti nella Tabella 4.

Dimensione	α Krippendorff	κ_w	ρ Spearman	Accordo ± 1
Naturalezza dialettale	0,767	0,774	0,778	90,7%
Accuratezza grammaticale	0,806	0,814	0,815	90,7%
Coerenza lessicale	0,781	0,775	0,790	86,7%

Tabella 4: Accordo fra i due annotatori sulle tre dimensioni della scala Likert, calcolato sui 150 item di ciascuna scheda.

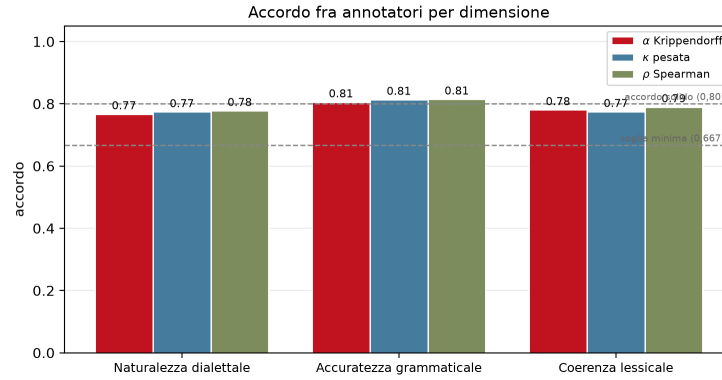


Figura 34: Accordo fra i due annotatori sulle tre dimensioni della scala.

La Figura 34 affianca i tre indici per ciascuna delle tre dimensioni, su un asse verticale che copre l'intero intervallo da 0 a 1 e con due linee orizzontali tratteggiate poste alla soglia minima convenzionale di 0,667 e al livello di accordo solido di 0,80. Tutte e nove le barre superano la prima linea, e le tre dell'accuratezza grammaticale sono le uniche a raggiungere la seconda, con 0,806, 0,814 e 0,815; naturalezza dialettale e coerenza lessicale si collocano poco al di sotto, fra 0,767 e 0,790.

Due letture emergono dal grafico. La prima è che i tre indici, costruiti su assunzioni fra loro diverse, restituiscono per ogni dimensione valori coincidenti: il livello di accordo osservato non è quindi l'artefatto di una particolare scelta di misura. La seconda è che l'ordinamento fra le dimensioni è quello atteso, dato che la grammatica ammette un giudizio più verificabile, mentre la naturalezza e soprattutto la coerenza lessicale chiamano in causa valutazioni di registro e di pertinenza su cui due parlanti nativi possono legittimamente divergere. La quota di giudizi che si discostano di al più un punto segue lo stesso andamento: 90,7% sulle prime due dimensioni e 86,7% sulla coerenza lessicale.

6 Conclusioni

Metrica	Minerva7B	Llama7B	Gemma3-4B
<i>Traduzione (T1) — generazione reale su test</i>			
chrF++ (test)	71,05	36,90	74,78
BERTScore F1 (test)	0,909	0,748	0,922
Recall lessicale dialettale	0,794	0,384	0,821
Rapporto di lunghezza ($\rightarrow 1$)	0,991	0,45	1,003
<i>Completamento di dialogo (T2)</i>			
Likert media (umano: 4,51)	3,42	3,07	2,69
ctx_accuracy (test)	0,660	0,689	0,689
Accuratezza token (test)	0,422	0,404	0,386
<i>Generazione libera (T3)</i>			
Likert media (umano: 4,62)	3,83	2,70	2,02
ctx_delta (contestuale)	2,385	1,953	1,796
ctx_delta (greedy)	1,048	0,026	0,278

Tabella 5: Confronto sintetico dei tre modelli di base — Minerva-7B-instruct-v1.0, Llama-2-7b-chat-hf e gemma-3-4b-it — sui tre task. In grassetto il valore migliore di ogni riga; per il rapporto di lunghezza il migliore è quello più vicino a 1.

La Tabella 5 riassume il confronto fra i tre modelli e mostra che l’esito dipende dal task. Sulla traduzione il modello migliore è Gemma3-4B, che pur avendo meno della metà dei parametri degli altri due e la peggiore conoscenza iniziale del napoletano ottiene il chrF++ più alto (74,78), il BERTScore più alto (0,922) e generazioni indistinguibili da quelle umane per un classificatore avversariale. Sui due task generativi, invece, il primato passa a Minerva7B: sul completamento di dialogo raggiunge la media Likert più alta (3,42) e la migliore accuratezza a livello di token (0,422), e sulla generazione libera stacca nettamente gli altri due sia sulla valutazione umana (3,83 contro 2,70 e 2,02) sia sull’uso del contesto (ctx_delta 2,385).

Considerando l’insieme dei tre task, il modello che risponde meglio all’obiettivo del progetto è **Minerva7B**. La ragione è duplice. In primo luogo, il progetto non punta a un semplice traduttore, ma a un sistema conversazionale capace di completare e generare turni pertinenti al contesto: sono proprio i task T2 e T3, quelli in cui Minerva7B prevale, a misurare questa capacità, e su T3 la valutazione umana separa la sua superiorità dallo zero con margine statistico (Minerva – Gemma = +1,81, Minerva – Llama = +1,13, entrambi con intervallo che non comprende lo zero). In secondo luogo, il vantaggio di Minerva7B non è confinato a una singola metrica ma è concorde su misure eterogenee — naturalezza, grammatica, coerenza lessicale e uso del contesto — il che rende il risultato robusto rispetto alla scelta dello strumento di valutazione.

Il confronto chiarisce anche il ruolo delle due variabili che distinguono i modelli. La specializzazione linguistica del modello di partenza è determinante sui task generativi: Minerva7B,

addestrato sull'italiano, parte da una perplessità di 19,72 sul napoletano contro i 43,22 di Llama7B, e mantiene questo vantaggio fino alla valutazione umana. La taglia, al contrario, non è una variabile decisiva in assoluto: Gemma3-4B, con 4 miliardi di parametri, batte entrambi i modelli da 7 miliardi sulla traduzione, ma resta indietro sui task che richiedono di decidere autonomamente cosa dire. La lettura complessiva è quindi che la conversione di un contenuto già fornito richiede capacità inferiori rispetto alla generazione pertinente al contesto, e che su quest'ultima la competenza linguistica concentrata del modello di partenza resta il fattore che fa la differenza.

7 Sviluppi Futuri

Il lavoro presentato costituisce una prova di fattibilità dell'adattamento di modelli linguistici di taglia contenuta al dialetto napoletano, ma lascia aperte diverse direzioni di ricerca e di ingegnerizzazione che ne estenderebbero sia la fruibilità sia la validità sperimentale. Se ne individuano in particolare tre, ordinate dalla più immediata sul piano applicativo alla più onerosa sul piano computazionale.

La prima direzione riguarda l'ingegnerizzazione di un'interfaccia conversazionale che renda il sistema effettivamente interrogabile da un utente finale. Allo stato attuale i tre adapter risultanti dal fine-tuning sono utilizzabili soltanto in un contesto di valutazione batch, in cui prompt e riferimenti sono predisposti a monte; l'obiettivo è invece esporli attraverso una *web application* che replichi il paradigma di interazione ormai consolidato dagli assistenti conversazionali, con una singola barra di immissione testuale in cui l'utente formula la richiesta e un'area di dialogo in cui la risposta viene restituita in tempo reale. Sul piano architetturale ciò richiede la separazione fra un *backend* di *inference serving*, incaricato di caricare il modello di base quantizzato e di innestarvi dinamicamente l'adapter LoRA corrispondente al task selezionato (traduzione, completamento o generazione libera), e un *frontend* leggero che comunichi con esso mediante un'interfaccia di programmazione REST, gestendo lo *streaming* dei token generati e il mantenimento della cronologia conversazionale necessaria a popolare la finestra di contesto dei task T2 e T3. Una simile interfaccia non avrebbe unicamente valore dimostrativo: consentirebbe di raccogliere interazioni reali e giudizi impliciti degli utenti, trasformando il sistema da artefatto valutato *offline* a piattaforma capace di alimentare iterativamente il proprio stesso addestramento.

La seconda direzione riguarda il principale vincolo incontrato in questo lavoro, ovvero la limitata dotazione hardware a disposizione. L'intero addestramento è stato condotto su una singola GPU Tesla T4 con 15,6 GB di VRAM, condizione che ha reso obbligata l'adozione di QLoRA con quantizzazione a 4 bit e ha di fatto precluso sia il *full fine-tuning* dei pesi sia l'impiego di modelli di base di taglia superiore ai 7 miliardi di parametri. Uno sviluppo naturale consiste nell'eseguire l'intera pipeline su un'infrastruttura locale dotata di acceleratori con maggiore memoria dedicata, eventualmente in configurazione multi-GPU, così da poter sperimentare modelli nell'ordine dei 13 o dei 70 miliardi di parametri e valutare in che misura la maggiore capacità del modello di partenza si traduca in un incremento della naturalezza dialettale e della coerenza lessicale, che la valutazione umana su scala Likert ha indicato come le dimensioni più critiche. Disporre di

risorse di calcolo adeguate permetterebbe inoltre di attenuare le semplificazioni imposte dall'ambiente Kaggle, come il sottocampionamento del set di sviluppo nelle valutazioni periodiche e la necessità di ripristinare i *checkpoint* fra sessioni successive, e di confrontare in modo rigoroso il *parameter-efficient fine-tuning* qui adottato con un aggiornamento completo dei parametri, quantificando l'effettivo costo prestazionale dell'approssimazione a basso rango.

La terza direzione riguarda l'ampliamento e l'arricchimento della base dati di addestramento. Per contenere i tempi di elaborazione, il presente lavoro si è limitato a due conversazioni interamente in italiano del modulo KIPasti, per un totale di 2589 righe, tradotte in napoletano e validate a mano; un tale volume, per quanto sufficiente a dimostrare l'efficacia della pipeline, resta esiguo rispetto alla variabilità lessicale e sintattica che il dialetto esprime nel parlato spontaneo. Uno sviluppo prioritario consiste pertanto nell'incrementare il numero di campioni, estendendo il corpus alle conversazioni campane già individuate nello stato dell'arte (in particolare le cinque registrazioni di Caserta (KPS021–KPS025), ricche di passaggi in dialetto autentico prodotti da parlanti nativi) così da affiancare al napoletano di traduzione, mediato dall'italiano, un napoletano di prima lingua non decontestualizzato. Un dataset più ampio e più eterogeneo consentirebbe di verificare empiricamente se le prestazioni migliorino in modo apprezzabile e, soprattutto, se la maggiore quantità di esempi mitighi il fenomeno di sovradattamento osservato in modo ricorrente sui task di completamento e generazione libera, in cui la perdita di validazione tende a risalire già dopo la prima o la seconda epoca. In prospettiva, l'ampliamento del materiale renderebbe percorribili anche tecniche di *data augmentation* controllata e di apprendimento curriculare, oggi impraticabili per la scarsità di dati, rafforzando la capacità del modello di generalizzare su contesti conversazionali non visti in addestramento.

Riferimenti bibliografici

- [1] Caterina Mauri, Silvia Ballarè e Eleonora Zucchini. *conversations.tsv — metadati delle conversazioni del corpus KIPasti*. <https://github.com/KIParla/KIPasti/blob/main/metadata/conversations.tsv>. GitHub repository, consultato il 20 agosto 2026. 2024.
- [2] Andrea Sapuppo. *Traduttore napoletano italiano online*. <https://www.scuolissima.com/2014/08/traduttore-italiano-napoletano-online.html>. Scuolissima.com, consultato il 20 agosto 2026. 2014.
- [3] Gemma Team. *Gemma 4 (31B) — gemma4:31b-cloud*. <https://huggingface.co/google/gemma-4-31B>
<https://ollama.com/library/gemma4:31b-cloud>. Hugging Face (Google DeepMind) e Ollama Cloud, consultato il 31 agosto 2026. 2026.
- [4] Sapienza NLP. *Minerva-7B-instruct-v1.0*. <https://huggingface.co/sapienzanlp/Minerva-7B-instruct-v1.0>. Hugging Face, Sapienza University of Rome, progetto FAIR (Future Artificial Intelligence Research), consultato il 29 agosto 2026. 2024.
- [5] Meta AI. *Llama-2-7b-chat-hf*. <https://huggingface.co/meta-llama/Llama-2-7b-chat-hf>. Modello disponibile su Hugging Face, consultato il 29 agosto 2026. 2023.

- [6] Google. *gemma-3-4b-it*. <https://huggingface.co/google/gemma-3-4b-it>. Modello disponibile su Hugging Face, consultato il 29 agosto 2026. 2025.